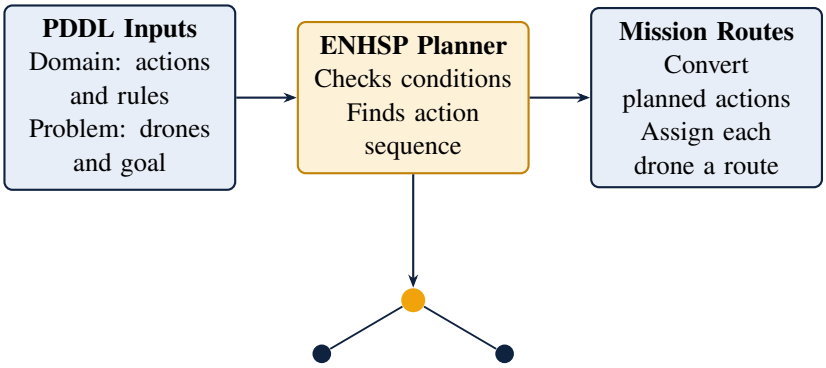


DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
INDIAN INSTITUTE OF TECHNOLOGY MADRAS
CHENNAI – 600036

PDDL-Based Mission Planning for a Drone Swarm Simulator



Drone Swarm Simulator

A Thesis

Submitted by

Vadithe Sai Kumar Naik

For the award of the degree

Of

MASTER OF TECHNOLOGY

December 2026

Thesis Certificate

This is to certify that the thesis entitled **PDDL-Based Mission Planning for a Drone Swarm Simulator**, submitted by **Vadithe Sai Kumar Naik (CS25M053)** to the Indian Institute of Technology Madras in partial fulfilment of the requirements for the degree of **Master of Technology**, is a bona fide record of the work carried out under the guidance of **Dr. N. S. Narayanaswamy**, Project Advisor, and **Dr. Jaimandeep Singh**, Project Co-Advisor.

The contents of this thesis, in full or in part, have not been submitted to any other institute or university for the award of a degree or diploma.

Chennai 600036
Date: December 2026

Vadithe Sai Kumar Naik
CS25M053

Dr. N. S. Narayanaswamy
Project Advisor, Professor
Department of Computer Science and Engineering
Indian Institute of Technology Madras

Dr. Jaimandeep Singh
Project Co-Advisor
Department of Computer Science and Engineering
Indian Institute of Technology Madras

© 2026 Indian Institute of Technology Madras

Acknowledgements

I would like to express my sincere gratitude to my project advisor, **Dr. N. S. Narayanaswamy**, and my project co-advisor, **Dr. Jaimandeep Singh**, for their valuable guidance, patient support and encouragement throughout this project. Their suggestions helped me develop clear PDDL models, understand the planning results and improve the presentation of this work.

I thank the Department of Computer Science and Engineering at the Indian Institute of Technology Madras for providing the facilities and academic environment needed to carry out this project.

I am grateful to my project teammates for their cooperation and useful discussions while integrating mission planning with the drone swarm simulator. Working together also helped me contribute to parts of the graphical interface and understand how a planned mission becomes a simulated flight.

Finally, I thank my family and friends for their constant encouragement and support during the project and the preparation of this report.

Vadithe Sai Kumar Naik
CS25M053
December 2026

Abstract

Keywords: automated planning; PDDL; numeric planning; ENHSP; drone swarm; formation flight; fault handling; battery management.

A drone swarm mission is more than a route. Before a drone may take off, join a formation or continue a leg, its battery must cover the next move, its GPS and radio links must work, its health must be acceptable and its neighbours in the formation must be in place. This thesis describes the planning layer of a software-in-the-loop drone swarm simulator, in which these rules are written in the Planning Domain Definition Language (PDDL) and solved by the Expressive Numeric Heuristic Search Planner (ENHSP). In this work, I developed the PDDL mission-planning models and contributed to parts of the graphical interface used to prepare missions.

Ten mission plans were developed, each as a domain file and a problem template: point-to-point travel, forest area search, V formation, two-dimensional grid formation, three-dimensional (cube) grid formation, parabola formation, figure-eight train formation, star drone show, planetary orbit show and road inspection. Every plan shares one vocabulary of drone states, health flags, failure states and numeric fluents for battery, energy, altitude and health. A plan service retargets each template onto the points the operator picks on the map, expands it to the number of selected drones and passes it to ENHSP. Only the leader corridor is usually planned symbolically; the other drones' routes are then derived geometrically from their formation slots.

Inter-drone communication is modelled in two layers. In PDDL, each formation declares which drones are neighbours and every formation action checks a per-drone `neighbor-comm-ok` flag; the parabola plan goes further and requires each directed link to be established within a communication range. At run time the ground station talks to each drone over a JSON-over-UDP protocol with heartbeats, and formation keeping for trains and shows runs on board each drone against a shared, time-stamped pace. Four fault types are modelled in the domains – GPS loss, communication loss, neighbour-link loss and health failure – together with ground-controller responses such as return, hover and sacrifice. Battery health is handled both as a planning resource (energy per metre, climb and leg energy, a 30-unit reserve) and at run time through low (20 %) and critical (5 %) thresholds that pause the swarm and ask the operator to decide.

The mission templates are adapted to the selected number of drones wherever the mission supports expansion. Travel accepts one or more drones, search at least two, V formation odd counts from three, grid formation at least four and cube formation at least eight. Cube planning has been demonstrated up to 64 drones; this is an observed example, not a general upper bound. Figure-eight capacity depends on the chosen path: the illustrated configuration supports up to 27 drones. The star show supports 6–40 drones and the planetary show supports 2–8. The current parabola mission uses exactly three active drones, and road inspection uses one. These mission-specific limits make clear where dynamic swarm sizing is supported and where the present implementation remains fixed. Planning cost, communication assumptions and the absence of automatic replanning after a failure remain limitations of this work.

Contents

Page

ACKNOWLEDGEMENTS i

ABSTRACT ii

LIST OF TABLES v

LIST OF FIGURES vi

ABBREVIATIONS vii

1 Introduction . . . 1

1.1 Background and Motivation . . . 1

1.2 Problem Statement and Objectives . . . 1

1.3 Scope and Contributions . . . 2

1.4 Organisation of the Report . . . 2

2 Background and Literature Review . . . 3

2.1 Automated Planning . . . 3

2.2 The Planning Domain Definition Language . . . 3

2.3 Numeric Planning with ENHSP . . . 4

2.4 Multi-Drone Missions and Formations . . . 4

2.5 Fault-Aware Planning for UAVs . . . 4

2.6 Position of This Work . . . 4

3 System Architecture and Planning Pipeline . . . 5

3.1 Simulator Overview . . . 5

3.2 From Map Clicks to Flying Drones . . . 5

3.3 Plan Kinds and Folders . . . 6

3.4 Run Records . . . 7

4 PDDL Modelling Approach . . . 8

4.1 A Shared Vocabulary . . . 8

4.2 The Common Mission Life Cycle . . . 9

4.3 Design Patterns for Tractable Swarm Models . . . 9

4.4 Understanding the ENHSP Result . . . 10

5 The Mission Plans . . . 11

5.1 Point-to-Point Travel . . . 11

5.2 Forest Area Search . . . 13

5.3 V Formation . . . 14

5.4 Two-Dimensional Grid Formation . . . 15

5.5 Three-Dimensional Cube Formation . . . 17

5.6 Parabola Formation . . . 18

5.7 Figure-Eight Train Formation . . . 20

5.8 Star Drone Show . . . 22
5.9 Planetary Orbit Show . . . 23
5.10 Road Inspection . . . 25
5.11 Swarm Size and Adaptability . . . 26
5.12 Summary of the Plans . . . 27

6 Inter-Drone Communication . . . 28
6.1 Communication in the PDDL Models . . . 28
6.2 The Range-Checked Link Model of the Parabola Plan . . . 29
6.3 Run-Time Communication between Ground Station and Drones . . . 29
6.4 On-Board Formation Keeping . . . 30
6.5 How the Two Levels Relate . . . 31

7 Fault Modelling and Battery Health . . . 32
7.1 Simple Fault Examples . . . 32
7.2 Faults Modelled in PDDL . . . 32
7.3 Ground Controller Responses . . . 33
7.4 Battery as a Planning Resource . . . 34
7.5 Battery Health at Run Time . . . 34
7.6 Fault Injection and Operator Decisions . . . 35

8 Verification and Results . . . 36
8.1 Method . . . 36
8.2 In-App Planning Requests . . . 36
8.3 Scalability of the Cube Formation . . . 36
8.4 Figure-Eight Capacity and Flight Quality . . . 37
8.5 Other Plans . . . 37
8.6 Reading a Parabola Plan . . . 38
8.7 Defects Found and Fixed . . . 38

9 Discussion and Limitations . . . 40
9.1 What the Models Check . . . 40
9.2 Limitations . . . 40

10 Conclusion and Future Work . . . 42
10.1 Conclusion . . . 42
10.2 Future Work . . . 42

References . . . 43

List of Tables

Table	Caption	Page
3.1	Simulator components	5
3.2	Mission inventory	7
4.1	Shared PDDL vocabulary	8
5.1	Swarm size and mission limits	27
5.2	Mission comparison	27
6.1	Formation neighbour relations	28
6.2	Run-time commands	30
7.1	Fault detection actions	33
7.2	Faults and run-time responses	35
8.1	Planning request records	36
8.2	Cube planning results	37
8.3	Figure-eight capacity	37
8.4	Other mission checks	38
8.5	Modelling fixes	39

List of Figures

Figure	Caption	Page
3.1	Planning pipeline	6
4.1	Common mission life cycle	9
5.1	Travel communication	12
5.2	Travel sample output	12
5.3	Search communication	13
5.4	Search sample output	14
5.5	V formation communication	15
5.6	V formation sample output	15
5.7	Rigid formation layouts	16
5.8	Grid formation communication	16
5.9	Grid formation sample output	17
5.10	Cube formation communication	18
5.11	Cube formation sample output	18
5.12	Parabola formation	19
5.13	Parabola formation communication	19
5.14	Parabola formation sample output	20
5.15	Figure-eight route	21
5.16	Figure-eight train communication	21
5.17	Figure-eight train sample output	22
5.18	Star show communication	22
5.19	Star show sample output	23
5.20	Star and planetary shows	24
5.21	Planetary show communication	24
5.22	Planetary show sample output	25
5.23	Road inspection communication	25
5.24	Road inspection sample output	26
6.1	Communication topologies	28
6.2	Ground-station messaging	29
6.3	Local formation control	31
7.1	Fault response sequence	34
7.2	Battery states and reserve	35
8.1	Cube planning scalability	37

Abbreviations

Abbreviation	Meaning
PDDL	Planning Domain Definition Language
ENHSP	Expressive Numeric Heuristic Search Planner
GCS	Ground Control Station
GUI	Graphical User Interface
UAV	Unmanned Aerial Vehicle
SYSID	MAVLink system identifier of a drone
MAVLink	Micro Air Vehicle Link, a lightweight drone messaging protocol
SITL	Software in the Loop
UDP	User Datagram Protocol
JSON	JavaScript Object Notation
OSRM	Open Source Routing Machine
AIBR	Additive Interval Based Relaxation (ENHSP preprocessing)
RTL	Return to Launch

Chapter 1

Introduction

1.1 Background and Motivation

Small unmanned aerial vehicles are now used for search, inspection, mapping and public light shows. Many of these tasks are done better by a group of drones than by one drone. A group, however, needs coordination. Each drone must know when to take off, which place in the formation it must hold, which other drones it must stay in contact with, and what to do when something goes wrong. Writing these rules directly in flight code makes them hard to read, hard to check and hard to change.

Automated planning offers a cleaner way. In planning, the rules of the world are written once as a *domain*: a set of actions, each with the conditions under which it may be applied and the changes it causes. A particular mission is written as a *problem*: the drones, the places, the starting state and the goal. A general-purpose planner then searches for a sequence of actions that reaches the goal while respecting every rule. PDDL is the standard language for writing such domains and problems.

This project is part of a larger drone swarm simulator built in the department. The simulator runs each drone as an independent thread and offers a ground station GUI with a map, telemetry and fault injection. The work reported here is the planning layer of that simulator: the PDDL models for every mission type, and their link to the rest of the system.

1.2 Problem Statement and Objectives

The central problem is to express swarm missions – including formations, communication requirements, battery limits and failures – in PDDL in a way that a numeric planner can solve quickly for realistic swarm sizes, and whose plans can be turned into flyable routes. The work had the following objectives:

1. Design a common PDDL vocabulary for drone state, health, communication, battery and failure that every mission plan can share.
2. Build domain and problem files for a range of missions: travel, area search, rigid formations (V, grid, cube, parabola), path-following trains (figure-eight), shape-changing shows (star, planets) and road inspection.
3. Model drone-to-drone communication, so that a formation can only form and fly while the required links are up.
4. Model faults (GPS, communication, neighbour link, health) and battery health, together with the ground controller's responses.

5. Integrate the plans with the simulator, so that a plan is retargeted to the operator's map inputs, solved by ENHSP and executed by the drones, and verify the results.

1.3 Scope and Contributions

In this work, I developed the PDDL mission-planning layer and contributed to parts of the graphical interface. The main planning contributions are:

- mission models for the supported tasks (Table 3.2), written using shared predicates and numeric fluents;
- a set of modelling patterns that keep formation problems tractable for large swarms: planning only the leader corridor, descriptive neighbour relations with per-drone communication flags, and propositional rather than numeric gates for ordered launches;
- a two-level model of communication: symbolic neighbour links in PDDL, including an explicit range-checked link model in the parabola plan, and the run-time protocol used by the ground station and the drones;
- a fault and battery model covering four failure types, ground-controller responses and energy accounting for travel, climb, cruise and hover;
- verification of every plan through the real planning pipeline, including scalability measurements and a record of the modelling defects found and fixed.

This report focuses on the PDDL models, their integration with mission preparation, and their verification in the simulator. My contribution also includes parts of the GUI. Physical flight tests are outside the scope of this work; the reported observations come from simulation.

1.4 Organisation of the Report

Chapter 2 reviews automated planning, PDDL, numeric planning and related work on multi-drone missions. Chapter 3 explains the simulator architecture and the planning pipeline. Chapter 4 presents the common PDDL vocabulary and design patterns. Chapter 5 describes all ten mission plans. Chapter 6 explains how drones communicate, both in the model and at run time. Chapter 7 covers fault modelling and battery health. Chapter 8 reports the verification results and the defects found. Chapter 9 discusses limitations, and Chapter 10 concludes with future work. Selected PDDL excerpts are explained alongside the corresponding missions in Chapter 5.

Chapter 2

Background and Literature Review

2.1 Automated Planning

Classical planning studies how an agent can reach a goal from a known initial state by applying actions with known effects. A planning task is described by a set of state variables, a set of actions and a goal. Each action has a *precondition* that must hold before it is applied and an *effect* that changes the state. A *plan* is a sequence of actions that, applied in order from the initial state, ends in a state that satisfies the goal. Heuristic search planners find such plans by estimating, for each state, how far it is from the goal and expanding the most promising states first.

2.2 The Planning Domain Definition Language

PDDL was introduced to give planning competitions a common input language. A PDDL model is split into two files:

- the **domain** defines object types, *predicates* (true/false facts such as `(gps-ok ?d)`), *functions* (numeric values such as `(battery ?d)`) and *actions*;
- the **problem** names the concrete objects, gives the initial facts and values, and states the goal.

PDDL 2.1 added numeric fluents, so that actions can test and change numbers, for example `(> (battery ?d) (energy-required ?from ?to))` as a precondition and `(decrease (battery ?d) ...)` as an effect. Later requirements such as `:conditional-effects`, `:universal-preconditions` and `:negative-preconditions` allow an action to say "for every wing drone, …" or "only if this drone is *not* already in formation". All of these features are used in this work.

2.3 Numeric Planning with ENHSP

ENHSP searches for a valid sequence of PDDL actions while also tracking numeric values such as battery charge and altitude. It first substitutes the actual drones and locations into the action templates; this is called grounding. It then uses a simplified estimate of the remaining work to guide the search. If it finds a plan, the action sequence satisfies the goal under the model's assumptions. A failed search does not by itself identify a faulty drone: the input may be infeasible, or search may have reached a time or memory limit.

In this project, conditions that compare a value with a fixed limit were easier to solve than some comparisons between two changing values. Large quantified formation models also increased grounding memory. These observations led to the modelling choices explained in Chapter 4.

2.4 Multi-Drone Missions and Formations

Aerial swarm research covers formation control, task allocation and collective behaviour. Formation flight is often treated with a leader–follower scheme, in which one drone follows the planned path and the others hold an offset from it. Early work on flocking showed that simple local rules with near neighbours are enough for coherent group motion. This project borrows both ideas: the planner reasons about a leader corridor, and each drone is required to stay in contact only with its immediate neighbours. For curved paths, shortest paths with a bounded turning radius were characterised by Dubins, and such paths are used to join the figure-eight pattern smoothly.

2.5 Fault-Aware Planning for UAVs

Real drones lose GPS, lose radio contact and run out of battery. A plan that ignores these events is not safe to fly. In planning, faults are commonly handled either by monitoring execution and replanning, or by including recovery actions in the domain. This work follows the second idea in a light form: each domain includes detection actions that turn a failed health flag into a failure state and a controller notification, and the point-to-point domain adds explicit controller orders (return, hover, sacrifice). Battery is treated as a numeric resource, so that the planner itself refuses any move the drone cannot afford.

2.6 Position of This Work

The contribution is not a new planning algorithm. It is a family of carefully engineered PDDL models for drone swarm missions, connected to a working simulator, together with an honest record of what the models express, how they scale and where they stop. The models are written to be read by people as well as solved by a planner, so that a supervisor or teammate can check every safety rule directly in the PDDL.

Chapter 3

System Architecture and Planning Pipeline

3.1 Simulator Overview

The simulator has four main parts. The **ground station GUI** (PySide6) shows the map, telemetry, the mission planner panel and fault injection controls. The **plan service** prepares PDDL problems and calls the planner. The **planner** is ENHSP, run as a Java process. The **drones** are independent threads, each with its own state, command inbox and telemetry output. Table 3.1 summarises the roles.

Table 3.1: Components of the simulator and their role in planning

Component	Responsibility
Mission planner panel (GUI)	Choose a named plan, pick source, destination, area or pattern points on the map, set laps and show speed.
Plan service	Select the plan folder, retarget the template to the picked points, expand it to N drones, call ENHSP, check the returned action order and build per-drone routes.
Problem preparation tools	Rewrite coordinates, distances and energy facts; clone template drones; build neighbour chains and geometry.
ENHSP wrapper	Ground and solve the problem; return the plan text, which is parsed into ordered steps.
Thread backend and GCS	Send missions and commands to each drone, receive telemetry and heartbeats, keep formation pace and raise low-battery, GPS-loss and drone-lost events.
Drone thread	Integrate motion and battery, monitor health and communication, fly the uploaded route.

3.2 From Map Clicks to Flying Drones

Figure 3.1 follows a mission from the map to simulated flight. The steps are:

Step 1: **Choose the mission.** I select the mission type, drones and required map points, such as a source, destination or search boundary.

Step 2: **Load its rules.** The service reads the domain and problem template for that mission. The domain describes allowed actions; the template supplies a starting example.

Step 3: **Use the selected map.** The service replaces example coordinates, recalculates route distances and energy costs, and inserts detours if needed.

Step 4: Use the selected swarm. It creates the required drone objects and initial facts, then rebuilds slots and neighbour relations. Mission-specific count and geometry checks must pass.

Step 5: Find a plan. ENHSP searches for an action sequence from the initial state to the mission goal. If no usable plan is returned, the mission is not uploaded.

Step 6: Read the result. The service parses the action sequence and checks that it contains the phases expected for that mission.

Step 7: Build individual routes. For leader-based formations, member routes are derived from the planned corridor and each drone's slot. Search and shows use their own route-generation rules.

Step 8: Execute and monitor. One mission is uploaded to each drone. The ground station displays progress and handles reported faults during flight.

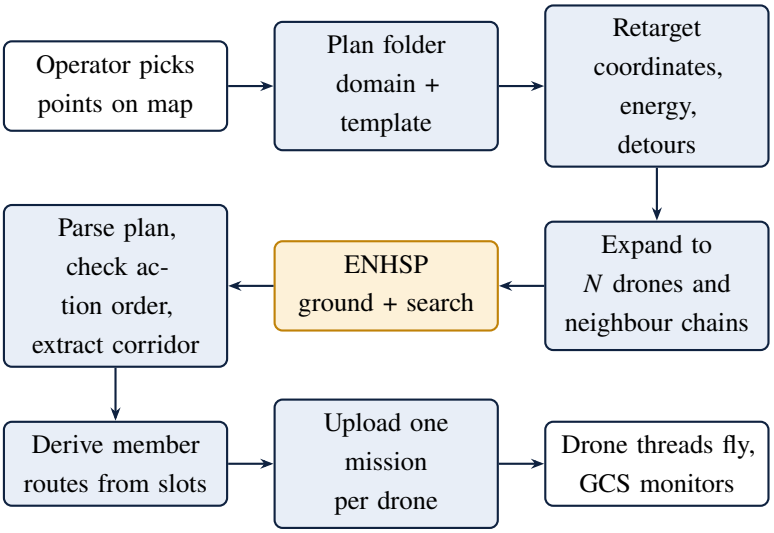


Figure 3.1: The planning pipeline, from the operator's map input to the drones' flight.

3.3 Plan Kinds and Folders

The plan service decides how to treat a plan from the *domain name* written inside `domain.pddl`, not from the folder name. This lets a folder be renamed freely and makes the domain file the single source of truth. Table 3.2 lists all ten plans.

Table 3.2: Inventory of the ten mission plans

Mission	Folder in plans/	Swarm size
Travel	`travell`	One or more
Search	`Search`	At least two
V formation	`vformation`	Odd count, at least three
Grid	`gridformation`	At least four
Cube	`3dgridformation`	At least eight
Parabola	`parabolaformation`	Exactly three active
Figure-eight	`figure8formation`	Two up to path capacity
Star	`starformation`	Six to forty
Planets	`planets`	Two to eight
Inspection	`Inspect`	One

3.4 Run Records

Every planning request writes a time-stamped folder under `plans/<name>/runs/` containing the exact problem that was solved and the full ENHSP output. These records made it possible to trace every unsolvable request back to its input, and they are the source of the statistics in Chapter 8.

Chapter 4

PDDL Modelling Approach

4.1 A Shared Vocabulary

The plans share names for basic drone states. A **predicate** is a true or false fact: `(gps-ok ?d)` means that drone `?d` has a GPS fix. A **numeric function** stores a value: `(battery ?d)` is the drone's remaining planning charge. The symbol `?d` is a placeholder for a particular drone; `?l` is a location. An action checks these facts before it runs and updates them afterwards. Table 4.1 groups the common symbols by purpose. They are model states, rather than network commands; the run-time commands are explained separately in Chapter 6.

Table 4.1: Shared predicates and numeric functions used across the domains

Symbol	Simple meaning
`(at ?d ?l)`	The drone is at this location.
`(landed ?d)`	The drone is on the ground.
`(airborne ?d)`	The drone has taken off.
`(at-cruise-altitude ?d)`	The drone has reached its assigned height.
`(safe-route ?a ?b)`	This route leg is allowed by the model.
`(gps-ok ?d)`	The drone has a usable position fix.
`(communication-ok ?d)`	Its link to the ground controller is assumed working.
`(neighbor-comm-ok ?d)`	Its required formation links are assumed working.
`(drone-healthy ?d)`	The drone passes the model health check.
`(controller-notified ?d)`	A failure notification has been recorded.
`(battery ?d)`	Remaining energy in planning units.
`(minimum-return-battery ?d)`	Energy reserve used by the relevant safety checks.
`(energy-required ?a ?b)`	Energy cost of this route leg.
`(target-altitude ?d)`	The height assigned to this drone.
`(slot-along-offset ?d)`	Its forward or backward offset from the leader.
`(slot-cross-offset ?d)`	Its sideways offset from the leader.

4.2 The Common Mission Life Cycle

Every formation begins with pre-flight checks at the source. Each drone must have working GPS and communication, acceptable health and enough battery. It takes off, climbs to its assigned cruise height and occupies its slot. The formation is established only when all required members are ready.

The leader then follows one route leg. A formation-specific close-up or hold action restores member positions and closes the formation before the next leg. This repeats until the destination or mission-specific final phase is reached. Figure 4.1 shows this order; Chapter 5 explains the extra rules for individual missions.

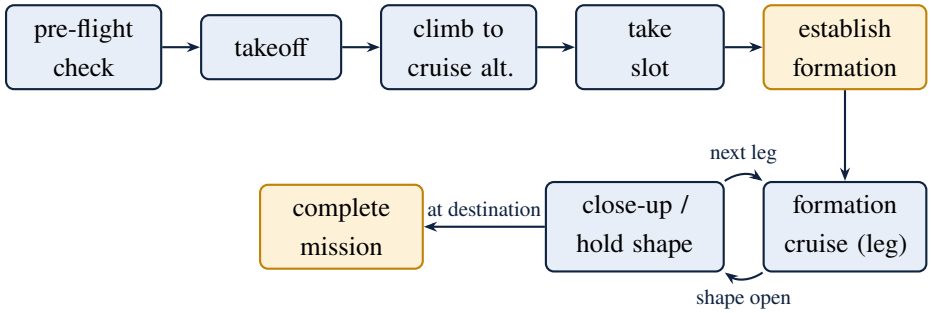


Figure 4.1: The life cycle shared by the formation plans. Each box is one PDDL action (or one action per drone).

The take-off step itself is short but illustrates the style used throughout:

```
(:action takeoff
    :parameters (?d - drone ?l - location)
    :precondition (and (at ?d ?l) (source ?l) (landed ?d)
                       (preflight-done ?d)
                       (gps-ok ?d) (communication-ok ?d) (drone-healthy ?d))
    :effect (and (not (landed ?d)) (airborne ?d) (flying ?d)
                 (assign (altitude ?d) (hover-altitude ?d))))
```

4.3 Design Patterns for Tractable Swarm Models

Plan the leader route. For a rigid formation, ENHSP chooses the leader's route. The service then builds the other routes by placing each drone at its assigned offset. This avoids asking the planner to search separately for every follower's geographic path.

Describe neighbours, check readiness. Neighbour facts record which drones belong next to each other. Most domains check a single readiness flag for each drone, rather than testing every link separately. The parabola domain checks individual directed links as well. A neighbour fact alone does not send a message or prove that a live radio link is working.

Launch cube layers in order. A layer is allowed to launch only after the layer below has reached its assigned height. Named layer states express this order more directly than comparing two changing counters. The rule reduces difficult search and keeps the launch sequence clear.

Keep template facts complete. The expansion service copies per-drone facts line by line. Each complete fact must therefore fit on one source line. Otherwise an added drone may receive an incomplete initial state, which can make a feasible mission appear unsolvable.

4.4 Understanding the ENHSP Result

ENHSP returns a sequence of grounded actions: each placeholder has been replaced by a named drone or location. Read the sequence from top to bottom. An action is possible only when its preconditions hold in the state produced by the previous actions. Its effects then create the state for the next step.

- **Action order** tells the service which mission phases are required. Leading step values in a sequential plan do not specify actual drone flight duration.
- **Plan length** counts symbolic actions. It is not route length or drone count.
- **Planning time** measures solving effort, rather than mission flight time.
- **Expanded and evaluated states** describe the search effort; they are not completed flights or success percentages.

A returned plan satisfies the written goal under the supplied initial conditions. It still needs parsing, mission-specific checks and route conversion. If ENHSP returns no plan, I first check the route, energy and formation inputs, then the solver's error, timeout or memory messages. A timeout does not prove the goal impossible. The next chapter shows a short travel result and explains the mission-specific facts that make each plan different.

Chapter 5

The Mission Plans

This chapter describes each of the ten plans: what the mission does, how it is picked on the map, the key facts of its model and the ordering of its actions.

5.1 Point-to-Point Travel

Mission. The whole selected swarm flies one shared route from a picked source to a picked destination, optionally around a marked no-fly area. The template's single drone `drone1` is cloned to every selected drone.

Model. This is the base domain, and it contains the richest fault handling (Chapter 7). Before each move a drone performs `check-distance` and `check-battery`; if the battery cannot cover the leg, `notify-insufficient-battery` records the fact instead. A route is planned only when every safety check passes:

```
(:action plan-safe-route
    :parameters (?d - drone ?from - location ?to - location)
    :precondition (and (at ?d ?from) (connected ?from ?to) (safe-route ?from ?to)
                       (battery-checked ?d)
                       (gps-ok ?d) (communication-ok ?d) (drone-healthy ?d))
    :effect (route-planned ?d))
```

The `travel` action moves the drone, sets it flying, decreases its battery by `energy-required` and clears `route-planned`, so the checks are repeated for every leg. A restricted direct route is marked with `restricted-route` and has no `safe-route` fact, so the planner must use the detour waypoints. A typical three-drone plan from the run logs is shown below.

Illustrative travel output (one drone, one leg)

```
0.0: (check-distance drone1 source destination)
1.0: (check-battery drone1 source destination)
2.0: (plan-safe-route drone1 source destination)
3.0: (travel drone1 source destination)
4.0: (reach-destination drone1 destination)
```

The leading values identify the order of actions in this sequential plan; they are not measured flight times. The first two actions check whether the leg and its energy requirement are acceptable. The route is then approved, travel changes the drone's model location and battery, and the final action records arrival. A longer route repeats the necessary leg checks. The meaning of solver statistics and the distinction between symbolic steps and flight time are explained in the previous chapter.

How the mission progresses. Each selected drone follows the approved route from source to destination. For each leg, it checks the route and available battery, records movement and finally records arrival. Adding drones repeats these checks for the additional drone objects; it does not introduce a formation slot.

Communication. Every active drone needs one ground-controller link. This domain declares no drone-to-drone neighbour relation, so its required peer-neighbour count is zero. The shared route does not by itself provide formation spacing or collision avoidance.

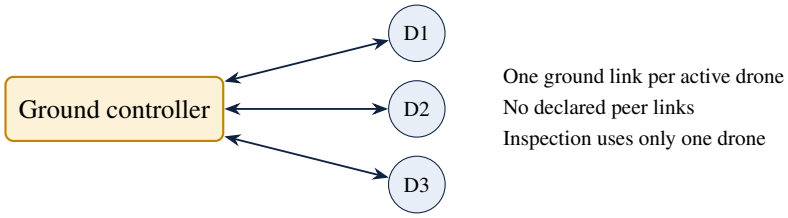


Figure 5.1: Travel communication. Double arrows show required model relations, not evidence of direct peer packets in the simulator. Every active drone also needs its ground-controller link.

Selected PDDL representation

```
(communication-ok drone1)
(connected source destination)
(safe-route source destination)
```

These initial facts say that drone1 has a controller link and that the direct leg exists and is allowed. The travel action still checks battery and health; a safe-route fact alone is not enough to permit flight.

Completion condition. Arrival at the destination and mission completion for every selected drone.

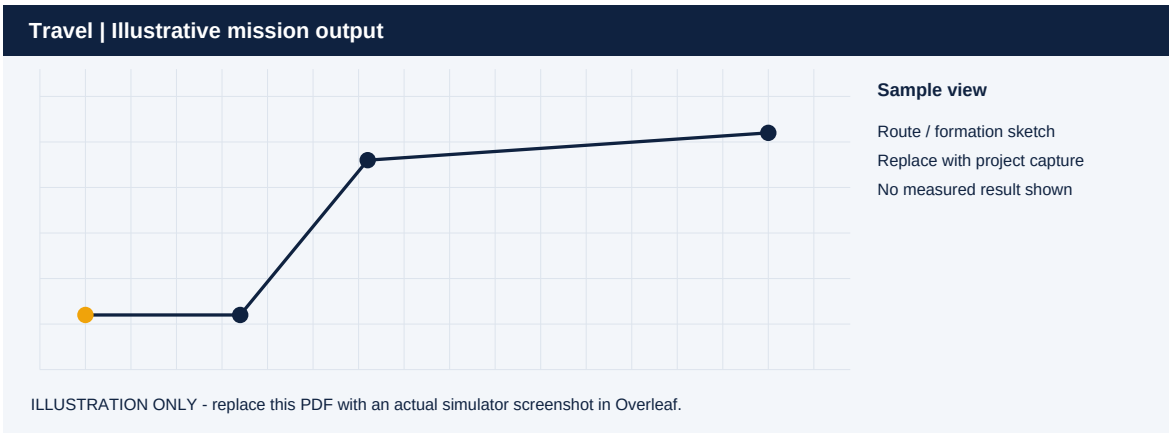


Figure 5.2: Travel sample output. This schematic is an illustration, not a recorded simulator result; replace it with the corresponding project screenshot.

5.2 Forest Area Search

Mission. A marked forest polygon is split into lanes, and each drone covers its own lane from a common base and returns. Unlike travel, each drone has its own route. At least two drones are needed.

Model. The type `forest-area` represents a lane, and `(assigned ?d ?a)` gives each drone its lane. The domain adds an explicit separation check between any two drones, using a pairwise numeric limit of 10 m:

```
(:action check-drone-separation
    :parameters (?d1 - drone ?d2 - drone ?l1 - location ?l2 - location)
    :precondition (and (at ?d1 ?l1) (at ?d2 ?l2) (not (= ?d1 ?d2))
                       (>= (distance ?l1 ?l2) (minimum-drone-separation ?d1 ?d2)))
    :effect (and (safe-separation ?d1 ?d2) (separation-ok ?d1) (separation-ok ?d2)))
```

A drone starts its search, moves through its assigned lane and marks the covered area. After the forest is covered, it returns to base and records completion. If its battery reaches the return reserve, an emergency-return action marks the drone as returning.

How the mission progresses. The selected polygon is divided into search lanes. Each drone leaves the base, visits the points assigned to its lane, marks its area covered and returns to the base. More drones divide the coverage work among more lanes, subject to valid area geometry and sufficient separation.

Communication. Each drone needs one ground-controller link. There is no declared peer communication topology in this domain: required peer neighbours are zero. Pairwise safe-separation facts describe spacing checks, rather than radio links.

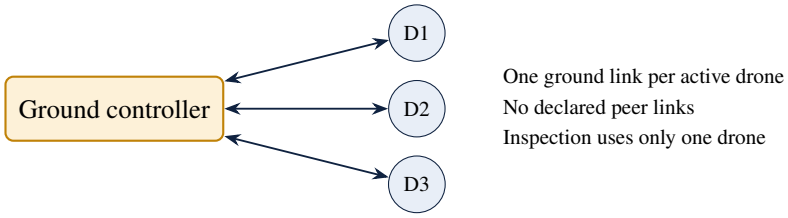


Figure 5.3: Search communication. Double arrows show required model relations, not evidence of direct peer packets in the simulator. Every active drone also needs its ground-controller link.

Selected PDDL representation

```
(assigned drone1 area1)
(assigned drone2 area2)
(communication-ok drone1)
(communication-ok drone2)
```

The assignments give the drones different areas. The communication facts represent their controller links. The model checks search progress and separation separately; assigning two lanes does not establish a peer link.

Completion condition. All assigned areas covered, all drone missions complete and drones back at base.

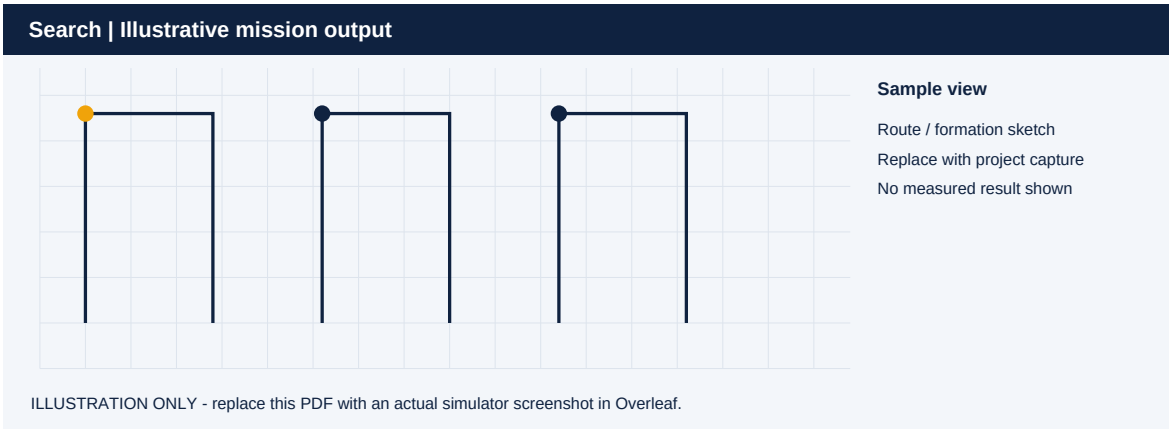


Figure 5.4: Search sample output. This schematic is an illustration, not a recorded simulator result; replace it with the corresponding project screenshot.

5.3 V Formation

Mission. Picked like travel. A leader (apex) flies the corridor and the other drones form two wings behind it. Each rank sits 17.32 m back and 20 m to the side of the one before it, so the wing angle is 60° (Figure 5.7a). Extra drones add further ranks.

Model. Wings launch after a short delay: `hold-position` lets the airborne leader wait, increasing `seconds-since-leader-airborne` until it reaches `wing-launch-delay`. Each wing drone takes its slot, and `establish-v-formation` requires every wing drone to be in formation, at the same location, at altitude and with `neighbor-comm-ok`. The pairwise fact `(wing-neighbor ?d1 ?d2)` records the chain leader–rank 1–rank 2 along each wing; the expansion code rebuilds this chain correctly for any larger odd drone count.

How the mission progresses. The leader takes off first and the wing drones launch after the required delay. Each wing drone moves into its assigned slot behind the leader. The leader then follows the route, and the wing positions are restored after each leg so that the V remains ready for the next move.

Communication. The leader has two wing neighbours, one on either side. An inner wing drone has two neighbours along its own wing; the last drone on a wing has one. In the smallest V, each wing drone talks only to the leader. These are declared model relations; run-time traffic is coordinated through the ground station.

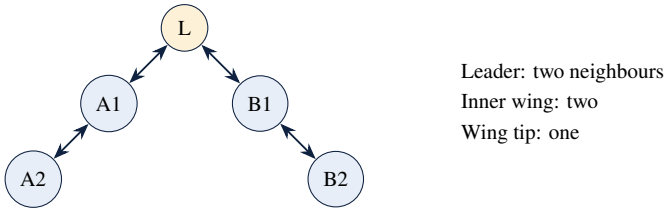


Figure 5.5: V formation communication. Double arrows show required model relations, not evidence of direct peer packets in the simulator. Every active drone also needs its ground-controller link.

Selected PDDL representation

```
(wing-neighbor drone-lead drone-left)
(wing-neighbor drone-left drone-lead)
(neighbor-comm-ok drone-left)
```

The two pair facts describe both directions of one wing link. The last fact summarises the left drone's neighbour readiness. Formation actions check readiness flags; they do not establish each pair as a live network connection.

Completion condition. The full formation reaches the destination and the formation mission is marked complete.

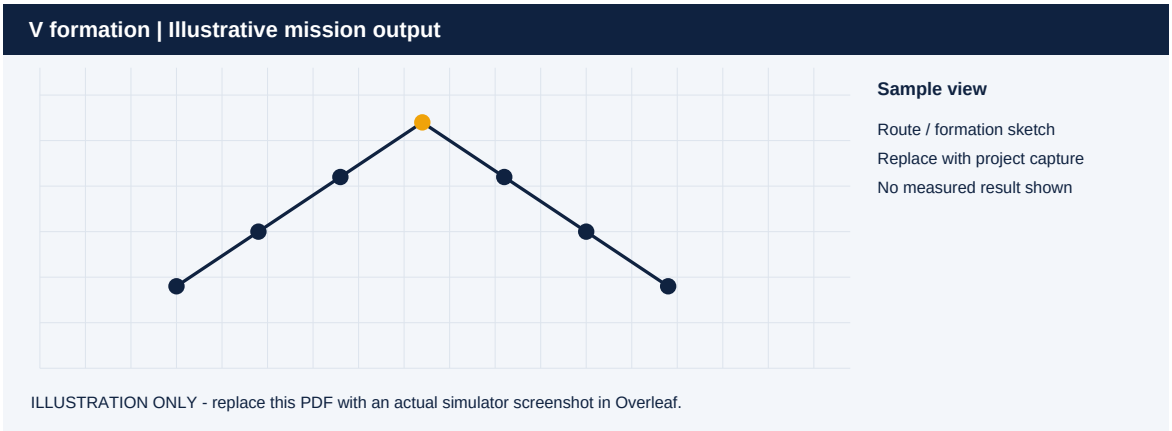
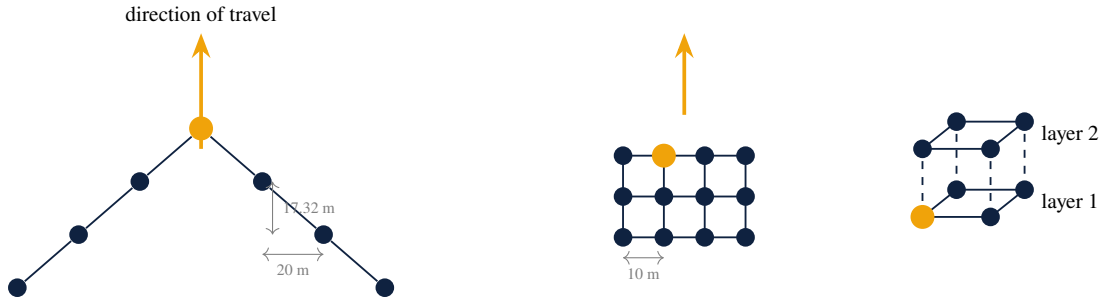


Figure 5.6: V formation sample output. This schematic is an illustration, not a recorded simulator result; replace it with the corresponding project screenshot.

5.4 Two-Dimensional Grid Formation

Mission. The drones fly as a flat grid, 10 m apart in every direction (Figure 5.7b). A square count gives a square grid; otherwise the most square rectangle that tiles the count exactly is used. At least four drones are needed.

Model. Each drone coordinates only with up to four neighbours (front, back, left, right), recorded by `grid-neighbor`. The leader's corridor is planned, and `close-up-grid` restores every member's cell after each leg. This domain needs ENHSP's naive grounder beyond about a dozen drones.



(a) V formation (b) 3×4 grid (c) $2 \times 2 \times 2$ cube

Figure 5.7: Rigid formations. The amber drone is the leader whose corridor ENHSP plans; lines show the neighbour links each drone must keep. In the cube, vertical layers are separated by height.

How the mission progresses. Drones take off, reach a common cruise height and occupy rectangular grid cells. The leader flies the corridor. After a leg, close-up-grid places members at the corresponding cells before the next leg begins. Increasing the swarm rebuilds the rows, columns and neighbour list.

Communication. A drone inside the grid has four neighbours: front, back, left and right. A boundary drone has fewer: normally three on an edge and two at a corner. A grid with a single row or column has at most two. Diagonal cells are not grid neighbours.

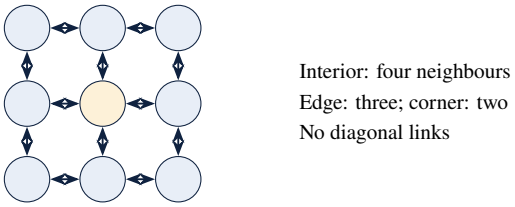


Figure 5.8: Grid formation communication. Double arrows show required model relations, not evidence of direct peer packets in the simulator. Every active drone also needs its ground-controller link.

Selected PDDL representation

```
(grid-neighbor drone-lead drone-member)
(grid-neighbor drone-member drone-lead)
(neighbor-comm-ok drone-member)
```

The pair facts identify adjacent cells in the template. During expansion, the service rebuilds these facts for the selected grid. The readiness flag is the condition checked by the formation actions.

Completion condition. Every member reaches the destination in the required grid mission state.

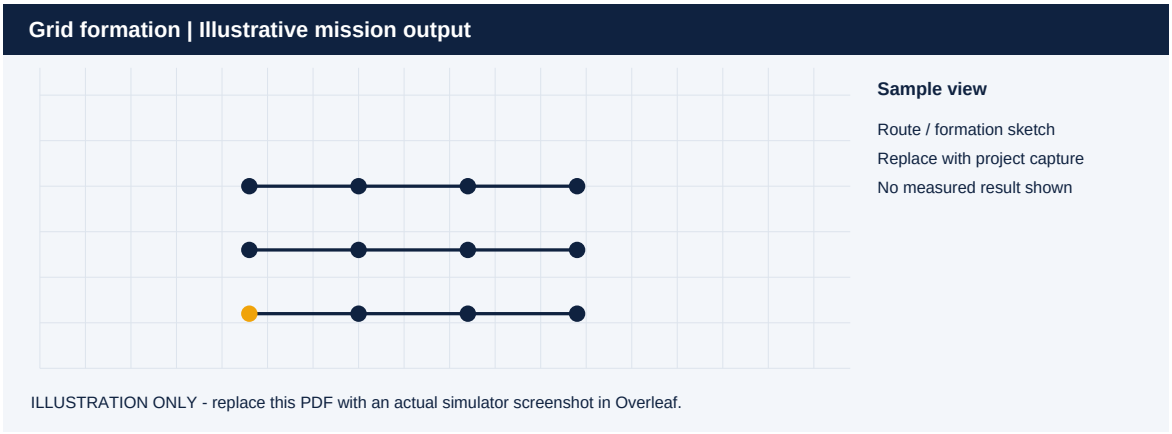


Figure 5.9: Grid formation sample output. This schematic is an illustration, not a recorded simulator result; replace it with the corresponding project screenshot.

5.5 Three-Dimensional Cube Formation

Mission. The grid extended into the vertical: a solid block of drones flies one corridor, 10 m apart horizontally and vertically (Figure 5.7c). A square footprint is chosen and as many layers as needed are stacked, allowing a partly filled top layer (for example 9 drones become $3 \times 2 \times 2$ with 9 of 12 cells filled; 25 become $3 \times 3 \times 3$). At least eight drones are needed.

Model. Layers are separated by height, so each drone has its own `target-altitude` (base plus 10 m per layer), and the run-time route adds a per-drone altitude offset. The swarm climbs layer by layer, bottom first:

```
(:action open-next-layer
    :parameters (?lo - layer ?hi - layer)
    :precondition (and (layer-below ?lo ?hi) (layer-launched ?lo)
                       (not (layer-launched ?hi))
                       (forall (?d - drone)
                           (or (not (in-layer ?d ?lo)) (at-cruise-altitude ?d))))
    :effect (and (layer-launched ?hi)
                 (forall (?d - drone)
                     (when (in-layer ?d ?hi) (cleared-to-launch ?d)))))
```

Each drone may coordinate with up to 24 neighbours: the $3 \times 3 \times 3$ block around it, except itself and the cells directly above and below. Collision avoidance is structural: fixed spacing, layer-by-layer ascent and a `cube-closed` interlock that must hold before each leg.

How the mission progresses. Drones are placed in rows, columns and height layers. The lowest layer launches and reaches its height before the next layer is released. Once all occupied cells are ready, the block follows the corridor and restores its shape between legs. A partially filled top layer is allowed.

Communication. A fully surrounded interior cell has up to 24 declared neighbours: the surrounding cells in a three-by-three-by-three block, excluding itself and the two cells directly above and below. Boundary cells and incomplete layers have fewer. This differs from the four-neighbour flat grid.

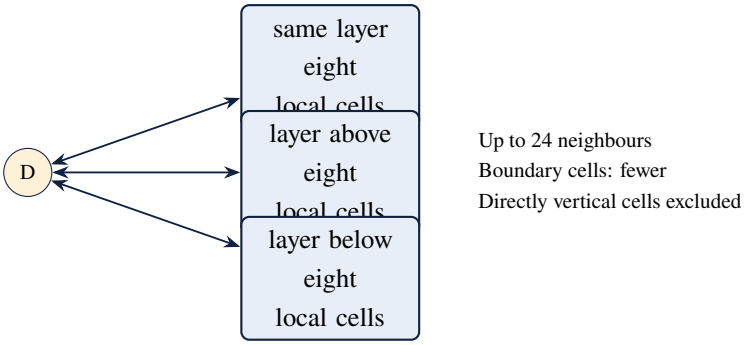


Figure 5.10: Cube formation communication. Double arrows show required model relations, not evidence of direct peer packets in the simulator. Every active drone also needs its ground-controller link.

Selected PDDL representation

```
(cube-neighbor drone-lead drone-member)
(cube-neighbor drone-member drone-lead)
(neighbor-comm-ok drone-member)
```

These are selected neighbour facts from the cube template. Expansion rebuilds the actual local cell links. Layer launch conditions and the cube-closed state provide separate ordering rules; the neighbour facts alone do not prevent collisions.

Completion condition. The occupied cube cells reach the destination with the formation complete.

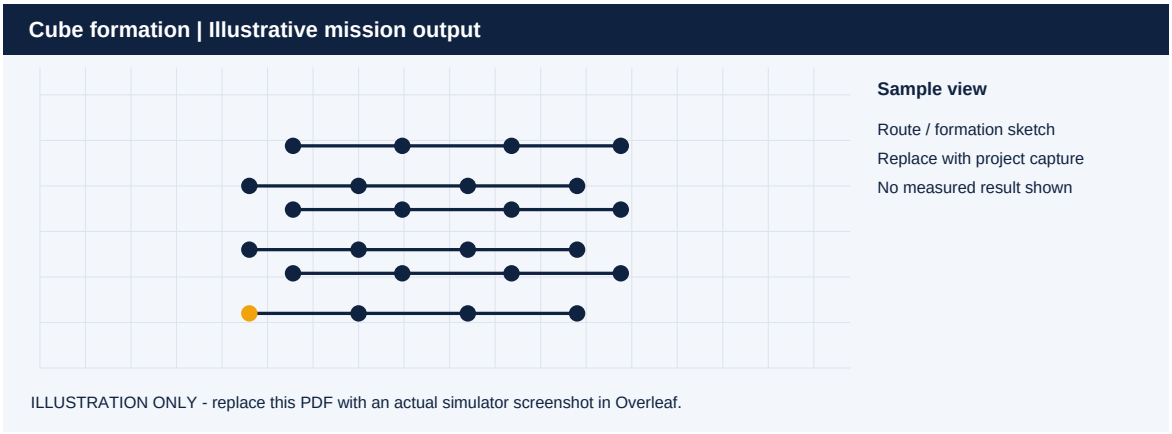


Figure 5.11: Cube formation sample output. This schematic is an illustration, not a recorded simulator result; replace it with the corresponding project screenshot.

5.6 Parabola Formation

Mission. Three drones fly side by side on one line across the route, 10 m apart. The middle drone (the vertex) cruises at 50 m and the two arm drones at 60 m, so seen from behind they form an upward-opening parabola (Figure 5.12). Extra selected drones stay on the ground.

Model. This plan has the most detailed communication model (Section 6.2). Before any drone may take its slot, every directed link must be established and each drone must confirm it is in contact with

all of its neighbours. The formation action checks the shape: each arm's target altitude minus the vertex's must equal `arm-rise` (10 m), so a flat line, an inverted U or lopsided arms give no plan, and every vertex–arm gap (14.14 m) must be at least `min-separation` (10 m).

```
(:action establish-parabola-formation
  :parameters (?vtx - drone ?l - location)
  :precondition (and (formation-leader ?vtx) (slot-vertex ?vtx) (in-formation ?vtx)
     (at ?vtx ?l) (at-cruise-altitude ?vtx) (swarm-comm-ready ?vtx)
     (forall (?a - drone)
       (or (not (arm-drone ?a))
           (and (in-formation ?a) (at ?a ?l) (at-cruise-altitude ?a)
                (= (- (target-altitude ?a) (target-altitude ?vtx)) (arm-rise))
                (>= (link-distance ?vtx ?a) (min-separation))
                (swarm-comm-ready ?a)))))
  :effect (and (parabola-established) (formation-at ?l) (shape-held)))
```

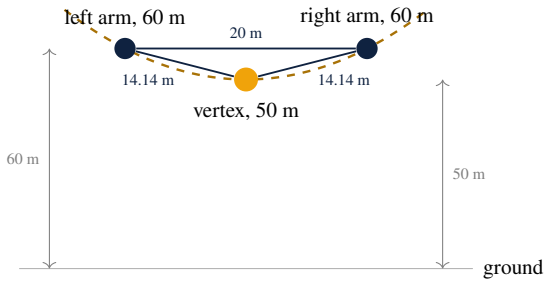


Figure 5.12: The parabola formation seen from behind. All three pairs keep a direct link (full mesh), and every link is within the 100 m communication range.

At run time the vertex corridor is split into legs of at most 200 m, and the arms are derived parallel at ±10 m. All three drones fly one shared timetable, so they arrive at each waypoint together even though the vertex has 10 m less to climb.

How the mission progresses. The vertex drone climbs to its lower assigned height, and the two arm drones climb to their higher heights. Each directed communication link is established, every drone confirms readiness, and only then may drones occupy their slots. The shape is checked before cruise and held again after each leg.

Communication. Each of the three active drones has two neighbours: the other two drones. Three drone pairs require six directed links because a link in one direction does not automatically create the reverse link. Extra selected drones remain on the ground in this implementation.

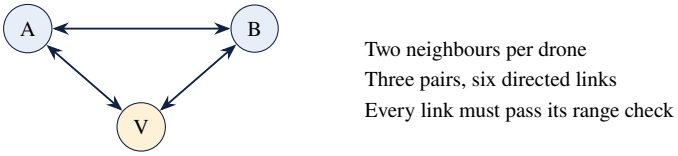


Figure 5.13: Parabola formation communication. Double arrows show required model relations, not evidence of direct peer packets in the simulator. Every active drone also needs its ground-controller link.

Selected PDDL representation

```
(parabola-neighbor drone-vertex drone-left)
(parabola-neighbor drone-left drone-vertex)
(link-up drone-vertex drone-left)
```

The first two facts declare the required pair in both directions. The last fact is an effect produced after establishing one directed link; it is not a substitute for the reverse link. Each link action checks range and radio readiness before setting link-up.

Completion condition. The three-drone shape reaches the destination and the parabola mission completes.

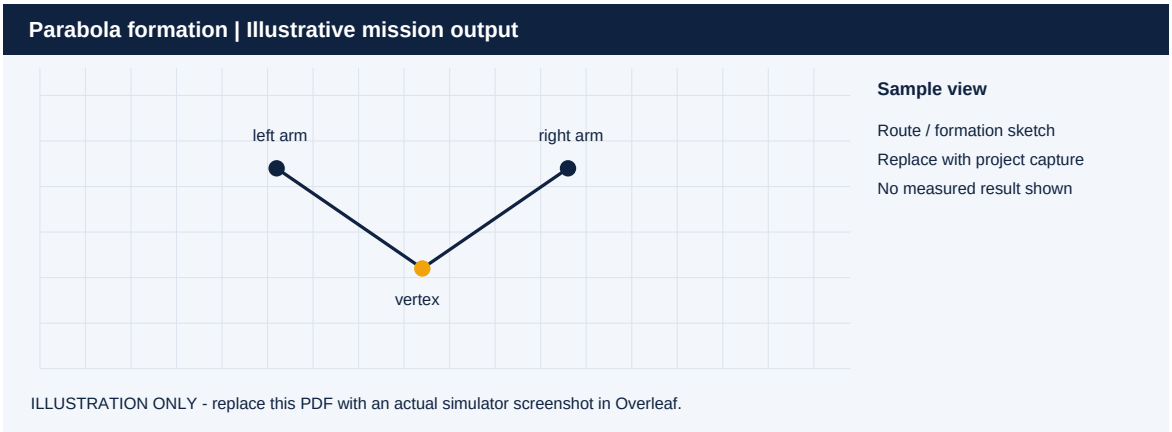


Figure 5.14: Parabola formation sample output. This schematic is an illustration, not a recorded simulator result; replace it with the corresponding project screenshot.

5.7 Figure-Eight Train Formation

Mission. The swarm flies a figure-eight in the air as a *train*: one drone behind another along the curve, 10 m apart. The operator makes four clicks: a base, then the left end, the crossing point and the right end of the eight. Each lobe is half a lemniscate of Gerono from the crossing to its clicked end, so the lobes cross at right angles and may differ in size (Figure 5.15). The drones fly from the base to the crossing along a Dubins path (turn radius at least 25 m), fly the requested number of laps, and return to their own landing pads.

Model. Each drone talks only to the drone directly in front and behind (`train-neighbor`). The leader's lap is planned through nine key waypoints with `formation-cruise`, and `follow-leader` moves the followers after every leg. Laps are counted numerically:

```
(:action start-next-lap
  :parameters (?lead - drone ?dest - location ?start - location)
  :precondition (and (lap-end ?dest) (lap-start ?start) (figure8-established)
                     (train-closed) (formation-at ?dest) (train-leader ?lead)
                     (at ?lead ?dest) (< (+ (laps-completed) 1) (laps-required)))
  :effect (and (not (formation-at ?dest)) (formation-at ?start)
               (not (at ?lead ?dest)) (at ?lead ?start)
               (increase (laps-completed) 1)))
```

Capacity. The only place the train can meet itself is the crossing. The train length $(N-1) \times 10\,\text{m}$ plus 14.1 m of crossing clearance must therefore fit inside the smaller lobe. This rule is checked in the GUI before planning, again in the plan worker, and inside PDDL itself as a precondition of `establish-figure8-formation`, so ENHSP refuses an over-full eight.

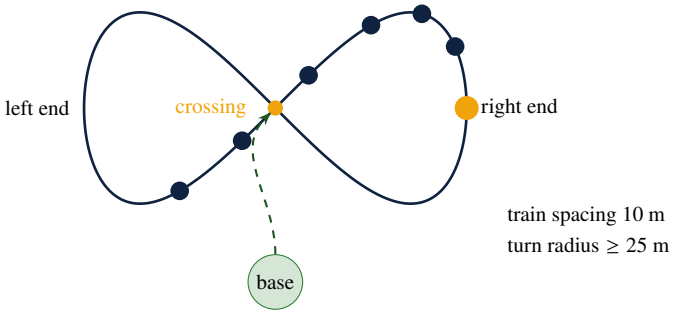


Figure 5.15: The figure-eight train. The leader (amber) and followers fly the same path one spacing apart; the base leg joins the eight tangentially at the crossing.

How the mission progresses. The drones join the figure-eight from the base and follow the same curve in order, with a fixed progress gap. The planner records lap progress; the local controller follows the shared pace. After the requested laps, each drone leaves the pattern and returns to its own landing pad.

Communication. The first and last drones in the ordered train have one neighbour each. Every middle drone has two: the drone immediately ahead and immediately behind. Crossing the same map position does not create an extra neighbour link. The path capacity check limits the total train length.

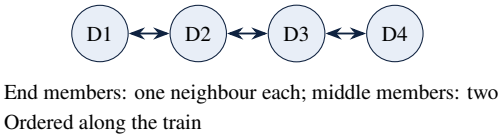


Figure 5.16: Figure-eight train communication. Double arrows show required model relations, not evidence of direct peer packets in the simulator. Every active drone also needs its ground-controller link.

Selected PDDL representation

```
(train-neighbor drone-lead drone-f1)
(train-neighbor drone-f1 drone-lead)
(neighbor-comm-ok drone-f1)
```

These facts describe consecutive train members, rather than all drones close to the crossing. The readiness flag permits formation actions. A shared run-time pace is supplied separately and is not a PDDL message exchange.

Completion condition. The required laps finish and the drones return to their assigned landing pads.

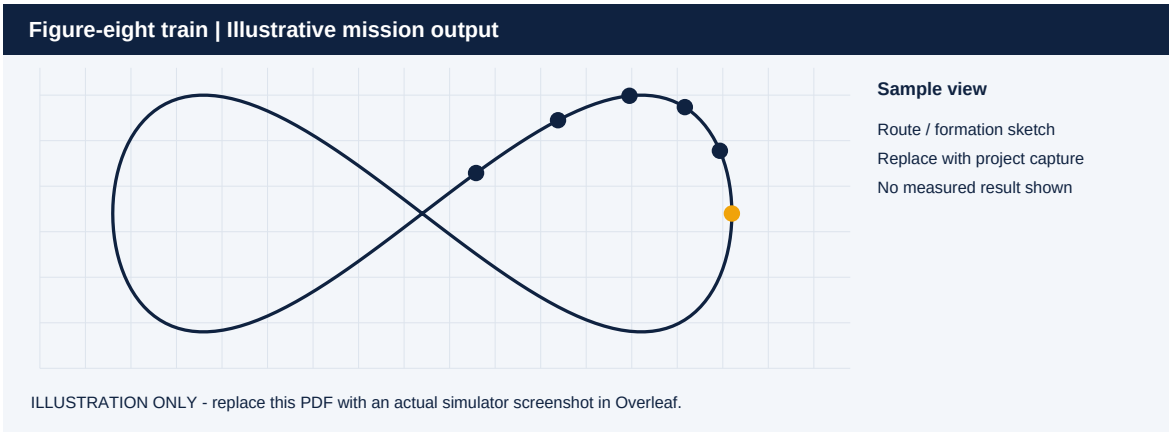


Figure 5.17: Figure-eight train sample output. This schematic is an illustration, not a recorded simulator result; replace it with the corresponding project screenshot.

5.8 Star Drone Show

Mission. A stationary show around one picked point. The drones take off from pads in rings, spread into a circle, and then, for each lap, form a star, return to the circle, spin the circle once, form the star again, spin the star and return to the circle, before landing on their own pads (Figure 5.20a). Six or seven drones make a 3-pointed star, eight or nine a 4-pointed star, and ten or more a 5-pointed star; 6 to 40 drones are allowed.

Model. ENHSP sequences `form-circle`, then per lap `form-star`, `form-circle-in-lap`, `spin-circle`, `form-star-again`, `spin-star` and `close-lap`, and finally `land-show`. The plan worker checks that exactly this sequence is returned. Every drone keeps the same order round the centre in the circle and the star, so shape changes never cross paths; the show is sized until all drones are at least 10 m apart in each shape and at least 9 m apart during transitions.

How the mission progresses. The drones rise from their pads and spread into a circle. For each lap, they form a star, return to a circle, rotate, form the star again and rotate it. Their order around the centre stays the same through the changes. After all phases, they return and land on their own pads.

Communication. Each drone has two ring neighbours, the preceding and following drone in cyclic order. The first and last entries are also linked, closing the ring. This relation remains meaningful when the visible shape changes from circle to star. All drones also receive the shared show clock through the ground station.

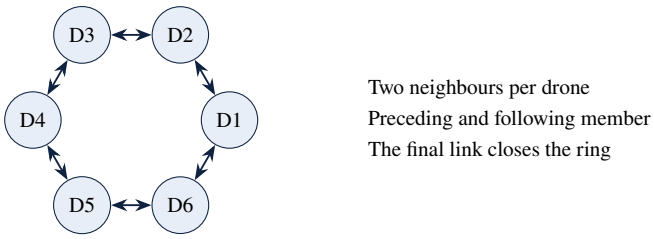


Figure 5.18: Star show communication. Double arrows show required model relations, not evidence of direct peer packets in the simulator. Every active drone also needs its ground-controller link.

Selected PDDL representation

```
(ring-neighbor drone-lead drone-m1)
(ring-neighbor drone-m1 drone-lead)
(neighbor-comm-ok drone-m1)
```

The pair facts are part of the cyclic ordering; expansion creates the complete ring for the selected swarm. The readiness flag is checked when forming the initial circle. Later show phases follow the planned phase order and shared schedule.

Completion condition. The requested show laps finish and every participating drone lands at the source.

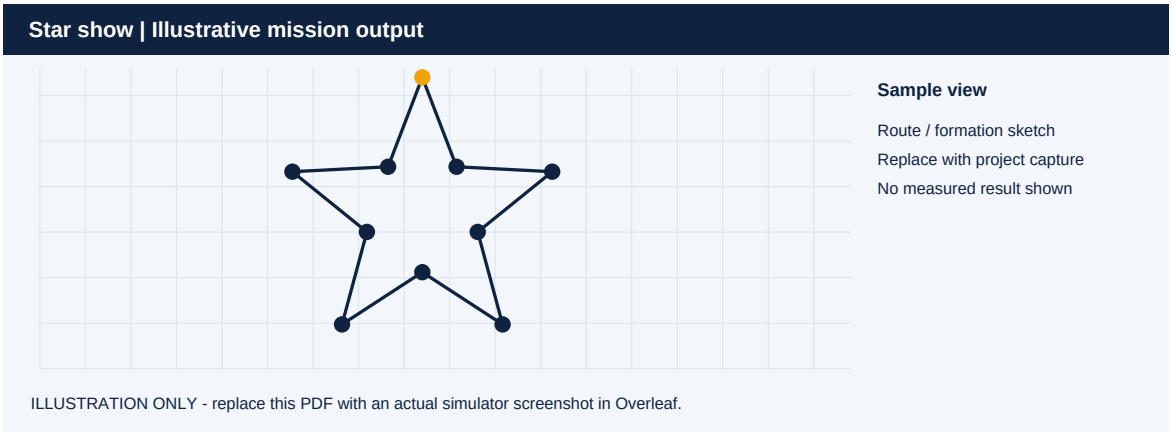


Figure 5.19: Star show sample output. This schematic is an illustration, not a recorded simulator result; replace it with the corresponding project screenshot.

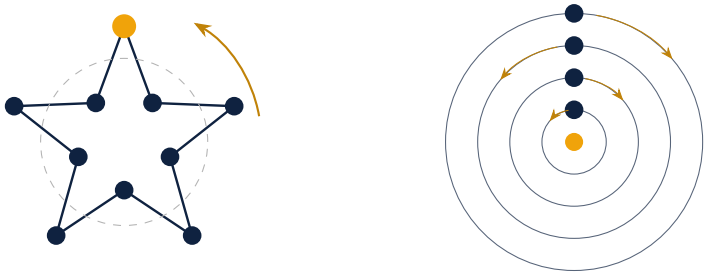
5.9 Planetary Orbit Show

Mission. A solar-system show round one picked point (the sun), with 2 to 8 drones, one per planet. Drone k orbits at a radius of $50(k + 1)$ m, and neighbouring orbits turn in opposite directions (Figure 5.20b). One lap is one turn of the outermost drone; inner drones make as many whole turns as fit, so all drones line up again on the north line at the end of every lap.

Model. `enter-orbits` may fire only when every drone has climbed and has `neighbor-comm-ok`, and only if every pair of orbit neighbours turns in opposite directions:

```
(forall (?a - drone ?b - drone)
    (or (not (orbit-neighbor ?a ?b))
        (and (orbits-clockwise ?a) (orbits-anticlockwise ?b))
        (and (orbits-anticlockwise ?a) (orbits-clockwise ?b))))
```

One `fly-lap` follows per lap, counted by `laps-completed`, then `land-planets`.

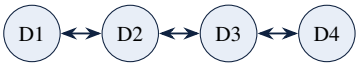


(a) star show, 10 drones (b) planets, alternating directions

Figure 5.20: Shape-changing shows. Both are flown against a shared timetable so that every phase starts and ends together for all drones.

How the mission progresses. Each drone enters its own circular orbit around the selected centre. Adjacent orbit indices have opposite rotation directions. A shared schedule keeps the show phases aligned; after the required laps, all drones return to their landing pads. Increasing the selected count adds orbits within the supported limit.

Communication. The innermost and outermost orbit drones each have one neighbour. Every intermediate orbit drone has two: the next smaller and next larger orbit. This is a chain ordered by orbit radius, rather than a complete mesh or a ring. With two drones, both have just one neighbour.



End members: one neighbour each; middle members: two
Ordered by orbit radius

Figure 5.21: Planetary show communication. Double arrows show required model relations, not evidence of direct peer packets in the simulator. Every active drone also needs its ground-controller link.

Selected PDDL representation

```
(orbit-neighbor drone-mercury drone-venus)
(orbit-neighbor drone-venus drone-mercury)
(orbits-clockwise drone-mercury)
(orbits-anticlockwise drone-venus)
```

The orbit pair requires opposite directions before enter-orbits is allowed. When more drones are selected, the service adds adjacent-orbit pairs and alternates their directions. Controller communication and neighbour readiness are checked separately.

Completion condition. All required laps finish and the participating drones land back at the source.

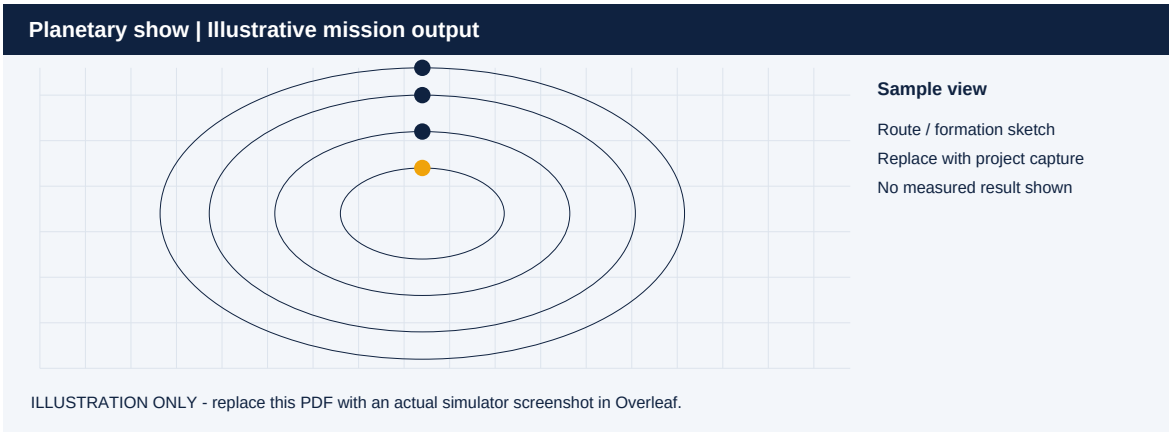


Figure 5.22: Planetary show sample output. This schematic is an illustration, not a recorded simulator result; replace it with the corresponding project screenshot.

5.10 Road Inspection

Mission. The operator clicks a source and a destination on a road. One drone takes off at the source, follows the road's exact path to the destination, flies the same road back and lands at the source. The road geometry comes from an OSRM routing server on OpenStreetMap data.

Model. The road is split into at most 12 segments, each with its own length and energy. ENHSP plans `inspect-segment` for each in order, `finish-inspection` at the destination, `return-along-road` back segment by segment, and `land-at-source`. Each segment inspection checks GPS, communication, health and that the battery covers the segment. If no road route exists, a click is more than 30 m from any road, or the two points are less than 20 m apart along the road, the plan fails before anything flies.

How the mission progresses. The road route is obtained from the selected source and destination and divided into ordered segments. The drone inspects each outbound segment, records that the road is inspected, follows the road back and lands at the source. It follows the road geometry rather than a straight line between the clicks.

Communication. There is one active inspection drone and one ground-controller link. Its required peer-neighbour count is zero because no second inspection drone or formation relation is created. Supporting a swarm inspection mission would require additional assignment and coordination rules.

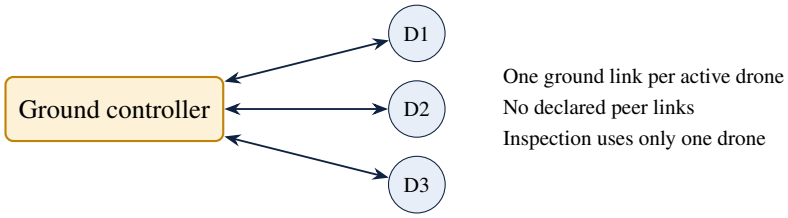


Figure 5.23: Road inspection communication. Double arrows show required model relations, not evidence of direct peer packets in the simulator. Every active drone also needs its ground-controller link.

Selected PDDL representation

```
(road-inspected)
(at drone1 source)
(landed drone1)
(mission-completed drone1)
```

These selected goal facts explain why reaching the far end alone is insufficient: the road must be inspected and the drone must return and land. The inspect-segment action separately checks the controller link and energy before each segment.

Completion condition. The road is inspected and the drone returns and lands at the source.

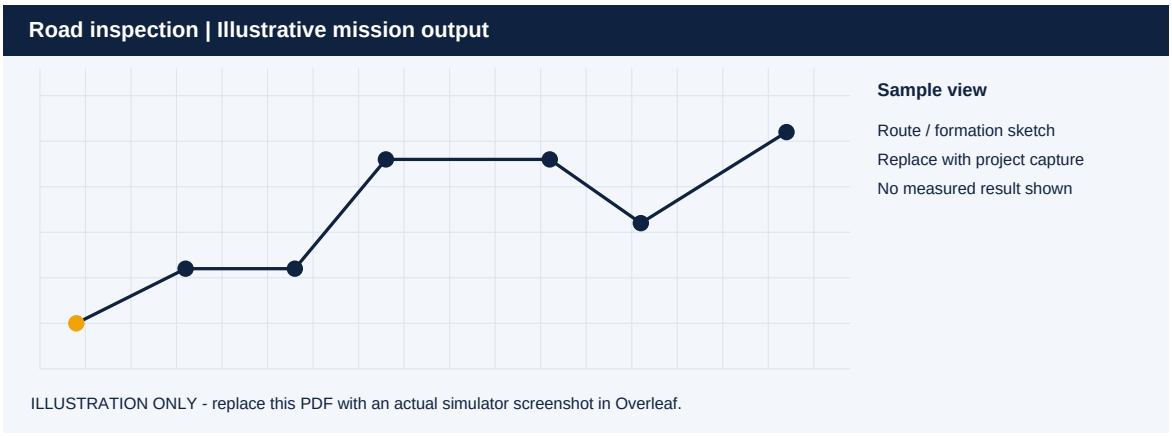


Figure 5.24: Road inspection sample output. This schematic is an illustration, not a recorded simulator result; replace it with the corresponding project screenshot.

5.11 Swarm Size and Adaptability

Selecting more drones rebuilds the expandable mission's objects, slots and neighbour relations. It does not make every mission unlimited. The limits below separate implementation constraints from examples already demonstrated in the source report. A demonstrated size is not a proof that every map at that size is feasible. Battery, route geometry and available solver memory still matter.

Table 5.1: Swarm size and mission-specific limits

Mission	Accepted size	Practical interpretation
Travel	One or more	No fixed upper limit stated; resource dependent.
Search	At least two	Lanes depend on polygon geometry and separation.
V	Odd count from three	Additional paired wing ranks; examples through seven.
Grid	At least four	Rebuilt rectangle; larger cases can exhaust grounding memory.
Cube	At least eight	Additional cells and layers; demonstrated through 64.
Parabola	Exactly three active	Vertex and two arms; additional drones stay grounded.
Figure-eight	Two to path capacity	Smaller lobe limits train length; example capacity 27.
Star	Six to forty	Implemented show-size range.
Planets	Two to eight	One drone per supported orbit.
Inspection	One	Current implementation has no swarm expansion.

5.12 Summary of the Plans

Table 5.2 compares the ten plans.

Table 5.2: Comparison of the mission plans

Plan	Map input	What ENHSP plans	Special feature
Travel	source, destination, no-fly area	every drone's legs	full controller fault responses
Search	base, forest polygon	one lane per drone	pairwise separation check
V	source, destination, no-fly area	leader corridor	timed wing launch
Grid	as V	leader corridor	4-neighbour links
Cube	as V	leader corridor	layer-by-layer ascent
Parabola	as V	vertex corridor	range-checked link mesh, shape check
Figure-8	base and three points	leader's lap	capacity rule, laps
Star	one point	show phases	circle/star/spin sequence
Planets	one point	show phases	opposite-direction orbits
Inspect	two points on a road	road segments	OSRM road geometry

Chapter 6

Inter-Drone Communication

Communication appears at two levels in the simulator. In the PDDL models it is a set of conditions: a drone may only act if its links are up. At run time it is a real exchange of messages between the ground station and the drones. This chapter explains both and how they relate.

6.1 Communication in the PDDL Models

Two flags carry communication state for every drone:

- `communication-ok`: the link between the drone and the ground controller works;
- `neighbor-comm-ok`: the drone's links to its formation neighbours work.

Formation establishment and movement actions check the relevant readiness conditions; exact checks vary by mission and phase. A formation cannot be established, cruise a leg or close up if any member has lost its neighbour links. Each plan also declares *who* must talk to whom, using a neighbour relation suited to its shape (Table 6.1, Figure 6.1).

Table 6.1: Neighbour relations declared by each formation

Plan	Relation	**Who talks to whom**
V formation	`wing-neighbor`	leader with rank 1, then rank to rank along each wing (a chain)
Grid	`grid-neighbor`	up to 4 neighbours: front, back, left, right
Cube	`cube-neighbor`	up to 24 neighbours in the surrounding $3 \times 3 \times 3$ block, excluding directly above and below
Parabola	`parabola-neighbor`	all three pairs, in both directions (6 directed links)
Figure-8	`train-neighbor`	the drone in front and the drone behind
Planets	`orbit-neighbor`	drones on adjacent orbits



(a) chain V wings, train — (b) 4-neighbour grid — (c) full mesh parabola — (d) ground link (all plans)

Figure 6.1: Communication topologies. (a)–(c) are drone-to-drone relations declared in PDDL; (d) is the ground-controller link represented by `communication-ok`.

6.2 The Range-Checked Link Model of the Parabola Plan

In most plans the links are summarised by the per-drone flag. The parabola plan models each directed link explicitly. A link from d_1 to d_2 can be established only when both drones are airborne at cruise altitude, both radios work, and the straight-line distance between them is within the communication range of 100 m:

```
(:action establish-comm-link
    :parameters (?d1 - drone ?d2 - drone)
    :precondition (and (parabola-neighbor ?d1 ?d2) (not (link-up ?d1 ?d2))
                       (airborne ?d1) (airborne ?d2)
                       (at-cruise-altitude ?d1) (at-cruise-altitude ?d2)
                       (communication-ok ?d1) (communication-ok ?d2)
                       (neighbor-comm-ok ?d1) (neighbor-comm-ok ?d2)
                       (<= (link-distance ?d1 ?d2) (comm-range)))
    :effect (link-up ?d1 ?d2))
```

A drone becomes `swarm-comm-ready` through `confirm-swarm-comm` once it has a link to every one of its neighbours (a universally quantified condition). Only then can it take its slot. During flight, `hold-parabola-shape` requires every declared link to still be up, and if a drone loses its neighbour links, `detect-neighbor-link-loss` removes all links to and from that drone, so the shape cannot be re-formed until they are restored. A negative test with the range reduced to 15 m (below the 20 m arm-to-arm distance) correctly produced no plan. In the solved plan, the six `establish-comm-link` actions and three `confirm-swarm-comm` actions appear right after the climbs and before any slot is taken (Section 8.6).

6.3 Run-Time Communication between Ground Station and Drones

At run time each drone is an independent thread that owns its state. It can only be reached through a command inbox and only reports through a telemetry callback. Over the network, the ground station and drones use a simple JSON-over-UDP protocol, one datagram per message, designed so that it can later be replaced by MAVLink. Every message carries the drone's MAVLink SYSID, so one ground socket can address the whole swarm (Figure 6.2).

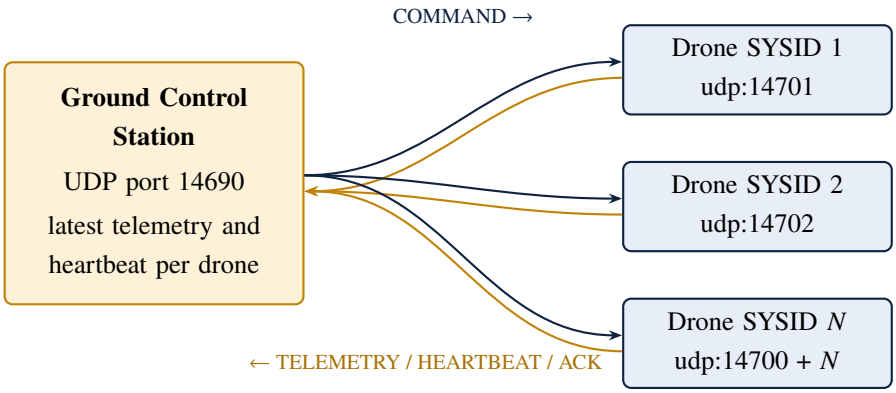


Figure 6.2: Run-time messaging. Commands are routed to a port derived from each drone's SYSID; every drone reports back to one ground station socket.

Table 6.2 lists the commands. A drone sends a heartbeat every second and telemetry every 0.5 s. The ground station treats a drone as out of contact if no heartbeat arrives for 3 s. On the drone side, if nothing is heard from the ground for more than 5 s the drone marks its link `DEGRADED`, and after 10 s `LOST`. Link loss is therefore *observed* from missing messages rather than assumed.

Table 6.2: Commands of the ground-station-to-drone protocol

Command	**Purpose**
`NAVIGATE`	The one mission operation: upload a whole route (approach, loop, laps, depart).
`ARM`, `DISARM`, `TAKEOFF`, `LAND`	Enable motors, disable motors, take off or land.
`HOLD`, `RESUME`	Pause and continue the mission (used while the operator decides on a fault).
`RETURN_TO_LAUNCH`, `ABORT`	Return to the launch point, or stop the mission.
`SET_GPS`	Fault injection: switch the drone's GPS fix off or on.
`SET_SPEED`	Change cruise speed in flight.
`TRAIN_PACE`	Formation keeping: the shared, time-stamped pace of the train or show clock.
`PING`, `SHUTDOWN`	Liveness check and stop the drone thread.

6.4 On-Board Formation Keeping

A formation needs a shared idea of progress. If the ground station repeatedly sends late speed corrections, a follower may speed up too much, brake too late and crowd the drone ahead. This work uses a shared pace and a small correction loop inside each simulated drone (Figure 6.3).

1. **Receive the pace.** The ground station sends the reference progress, reference speed and time stamp in a `TRAIN_PACE` message.
2. **Find the assigned position.** Each drone updates the reference for the elapsed time and subtracts its own distance behind the leader. For example, a follower assigned one spacing behind aims for one spacing less path progress.
3. **Compare with actual progress.** The drone measures how far it has travelled along its own route. If it is behind its target, it may increase speed; if it is ahead, it slows down.
4. **Respect the drone ahead.** A reported small front gap limits any speed increase. The ground station can slow the common pace for a lagging drone. This depends on reported information and does not guarantee collision avoidance.

For star and planetary shows, the reference is a shared show clock. Each drone has a timetable telling it where to be at each phase. Using the same clock helps all drones start and finish a shape change together. This local speed controller follows a prepared route; it does not create a new PDDL mission after a fault.

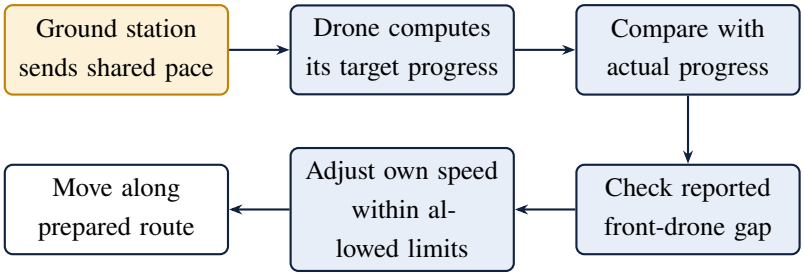


Figure 6.3: On-board formation keeping. Only the pace is sent from the ground; each drone closes its own gap every tick.

6.5 How the Two Levels Relate

The PDDL flags are a *planning-time* statement: the plan is only valid under the assumption that the listed links work. The run-time protocol is what actually carries commands and reports. The two are connected through the fault events of Chapter 7: when the run time observes a fault, the mission is held and the operator decides how to continue. The PDDL conditions do not by themselves monitor real radio links; this distinction is discussed in Chapter 9.

Chapter 7

Fault Modelling and Battery Health

7.1 Simple Fault Examples

A fault means that a condition needed for the mission is no longer true. These examples illustrate existing rules; they do not add unimplemented recovery behaviour.

- **GPS is lost.** A drone no longer has a reliable position fix. The GPS condition fails, so the model cannot continue its normal move. In the simulator, the operator can hold the mission and decide what to do next.
- **A controller message is missing.** Missing heartbeats can indicate lost contact. The ground station reports the problem; a neighbour fact in PDDL cannot restore the connection.
- **A formation neighbour is unavailable.** The formation no longer meets its assumed readiness conditions. The plan has no automatic action that redesigns the formation around the missing member.
- **Battery is low.** A proposed leg may need more energy than is available. Planning checks can reject the leg; during flight the low-battery warning asks the operator to decide whether to land.

The common sequence is: recognise the problem, stop relying on the failed condition, notify the controller, and choose an available response. Changing a PDDL state flag does not itself make a physical drone stop or repair the fault.

7.2 Faults Modelled in PDDL

Every domain includes the same family of detection actions. Each one fires for a flying drone whose health flag has failed, records a failure state, and takes the drone out of flight and, in formation plans, out of the formation. Table 7.1 lists them.

Table 7.1: Fault detection actions shared by the domains

Action	**Fires when**	**Effect**
detect-gps-loss	flying and not `gps-ok`	`gps-lost`; not flying; leaves the formation (travel also sets `gps-warning`)
detect-communication-loss	flying and not `communication-ok`	`communication-lost`; not flying; leaves the formation; parabola also clears `swarm-comm-ready`
detect-neighbor-link-loss	flying and not `neighbor-comm-ok` (formation plans)	`communication-lost`; leaves the formation; parabola also drops every link to and from the drone
detect-health-failure	flying and `health` < `minimum-health` (40)	`health-failure`; not flying; leaves the formation
notify-ground-controller	any of the three failure states	`controller-notified`

The GPS loss detection, for example, is:

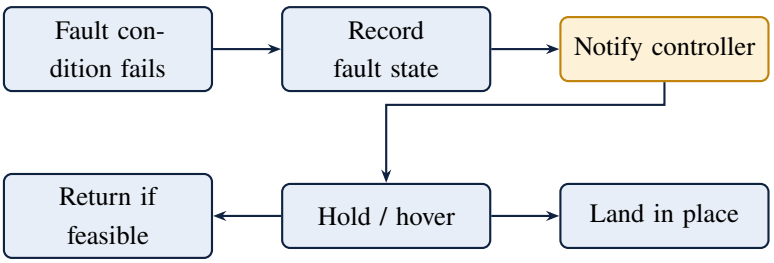
```
(:action detect-gps-loss
    :parameters (?d - drone)
    :precondition (and (flying ?d) (not (gps-ok ?d)))
    :effect (and (gps-lost ?d) (not (flying ?d)) (not (in-formation ?d))))
```

Because every movement and formation action also requires `gps-ok`, `communication-ok`, `drone-healthy` and, for formations, `neighbor-comm-ok`, a problem that starts with a failed flag has no plan that moves the affected drone. The planner thus refuses to fly an unhealthy drone, and a failed member blocks the formation actions that depend on it.

7.3 Ground Controller Responses

The point-to-point domain goes further and models what the ground controller can order once it has been notified (Figure 7.1):

- **Return**: `controller-order-return` sets `emergency-return`, and `return-to-source` flies the drone back over a safe route, provided the battery covers the return leg.
- **Hover**: `controller-order-hover` puts the drone in a hover. The `hover` action drains `hover-energy` while the battery stays above the return reserve; when it reaches the reserve, `initiate-low-battery-return` switches the drone to an emergency return.
- **Sacrifice**: `sacrifice-drone` lands the drone immediately where it is when the controller decides it cannot be recovered.



Controller chooses an available response.
No automatic replacement PDDL mission is generated.

Figure 7.1: Fault handling in the point-to-point domain: detection, notification and the three controller responses.

7.4 Battery as a Planning Resource

Battery is a numeric fluent in every domain, in model units with 100 as a full charge. It is used in three ways.

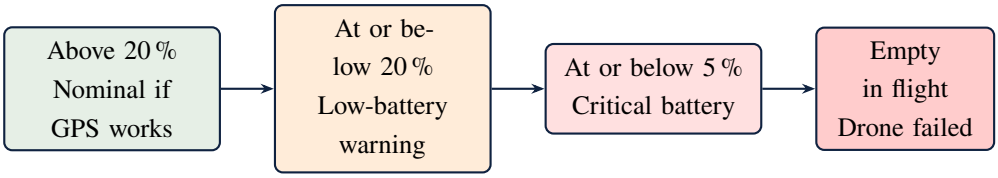
Energy per leg. Each leg has an `energy-required` value. When a template is retargeted, this value is recomputed from the new leg length at the leg's own energy-per-metre rate, with a default of 0.0125 units per metre (12.5 units per kilometre). Every move requires the battery to be strictly greater than the leg's cost and decreases it by that cost.

Climb, cruise and hover energy. Formation plans charge `climb-energy` for the climb to cruise altitude (in the parabola plan 4 units for the vertex and 5 for the arms, which climb 10 m further), and `cruise-leg-energy` (12 units) for each follower or arm leg. The travel domain charges `hover-energy` for each hover step.

Reserve for return. Each template uses a return reserve of 30 planning units. Pre-flight checks require enough charge before launch. The relevant leg checks decide whether the next move can be afforded; hover and search models include return behaviour when the reserve is reached. Health is a separate numeric score (100 initially, minimum 40), so a drone can be refused for poor health even with a full battery.

7.5 Battery Health at Run Time

During flight each drone thread integrates its own battery. With the default settings a drone has a 5 200 mAh battery and draws a nominal 20 A in flight; its percentage is computed from the consumed charge. The drone classifies its own health from the battery and the GPS fix (Figure 7.2): `NOMINAL` above 20 %, `DEGRADED` at or below 20 % or without GPS, `CRITICAL` at or below 5 %, and `FAILED` if the battery is exhausted in flight, in which case the drone is lost.



Planning uses a separate 30-unit return reserve.
Model units are not calibrated to run-time battery percentages.

Figure 7.2: Run-time battery states. A low-battery event pauses the mission for an operator decision. The PDDL reserve is a separate planning quantity, rather than a calibrated percentage threshold.

When any drone crosses the low-battery threshold, the ground station *pauses the whole mission* so that the drones do not drain while the operator decides; at the compressed time scale the decision window would otherwise be a fraction of a second. A dialog shows the drone's battery and the nearest landing point, with a note whether it is within reach on the remaining charge. The operator may divert and land now, or press on, in which case the drone will be lost when the battery runs flat. A drone whose battery reaches zero in flight is reported as lost and sacrificed.

7.6 Fault Injection and Operator Decisions

The GUI's fault injection panel lets the operator target one or more drones by SYSID with a fault type (GPS loss, communications loss or battery failure), a severity and a duration (zero meaning until reset). In the thread-based backend, GPS loss is delivered as a `SET_GPS` command, so the drone itself loses its fix. On GPS loss the ground station holds the swarm and asks the operator whether the drone should fall back to its start point and land, showing the distance back and whether the remaining charge covers it; otherwise the drone holds position until GPS returns, while its battery keeps draining. When GPS is reacquired the mission resumes. Table 7.2 maps each fault to its PDDL model and run-time behaviour.

Table 7.2: Faults: how each is modelled in PDDL and handled at run time

Fault	**PDDL model**	**Run-time behaviour**
GPS loss	not `gps-ok`; `detect-gps-loss`	injected with `SET_GPS`; drone `DEGRADED`; swarm held; operator chooses fall-back or hold
Communication loss	not `communication-ok`; `detect-communication-loss`	observed from missing heartbeats and contact (`DEGRADED` after 5 s, `LOST` after 10 s)
Neighbour link loss	not `neighbor-comm-ok`; `detect-neighbor-link-loss`	reflected in formation keeping through the front-drone cap and pace slowing
Health failure	`health` below `minimum-health`; `detect-health-failure`	drone health classes `NOMINAL/DEGRADED/CRITICAL/FAILED`
Low battery	battery not above reserve; `notify-insufficient-battery`, `initiate-low-battery-return`	at 20 % swarm paused, operator chooses emergency landing or press on; at 0 % drone lost

Chapter 8

Verification and Results

8.1 Method

Each plan was verified in two ways. First, ENHSP was run directly on problems produced by the real expand, retarget and solve pipeline, for a range of drone counts. Second, the resulting routes were flown without the GUI through the drone threads, and the spacing, altitudes and landing positions were measured. Negative tests were added where a plan must refuse a problem. The run logs in `plans/*/runs/` record the in-app requests.

8.2 In-App Planning Requests

Table 8.1 summarises the logged requests for the four longest-running plans. Every unsolvable travel and V request was traced to an infeasible combination of source, destination and no-fly area, where no safe path existed. The grid failures all occurred in a short window before a template fix (Section 8.7); every grid request after the fix solved.

Table 8.1: Logged planning requests per plan

Plan	Runs	Solved	Unsolvable	Notes
Forest search	69	69	0	no failures on record
Travel	83	76	7	infeasible no-fly inputs during early testing
V formation	65	62	2	2 infeasible inputs; 1 run closed mid-plan
Grid	19	15	4	all 4 before the hover-altitude fix

8.3 Scalability of the Cube Formation

Table 8.2 and Figure 8.1 show planning time and plan length for the cube formation. Plan length grows linearly with the number of drones, about four actions per drone. Planning time stays below 4 s up to 36 drones and reaches 50 s for a full $4 \times 4 \times 4$ block of 64 drones. A launch-order audit of the 25-drone plan found no drone taking off before its layer opened and no layer opening before the one below had finished climbing.

Table 8.2: Cube formation: ENHSP results through the real pipeline

Drones	8	9	12	16	25	27	36	64
Shape	2×2×2	3×2×2	2×3×3	2×3×3	3×3×3	3×3×3	3×4×4	4×4×4
Time (s)	0.4	0.5	0.6	0.7	1.2	1.8	3.4	50.0
Plan length	42	47	58	74	111	119	155	268

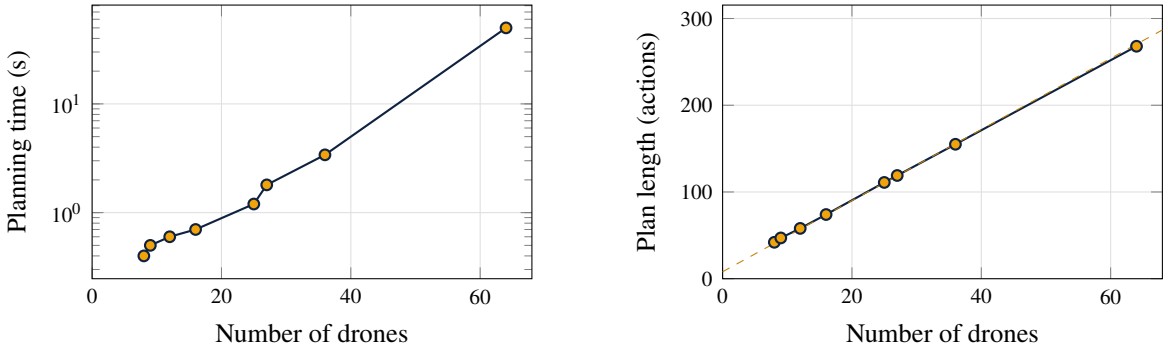


Figure 8.1: Cube formation scalability. Left: planning time (log scale). Right: plan length, close to linear in the number of drones (dashed line $\approx 4.1N + 8$).

8.4 Figure-Eight Capacity and Flight Quality

On an asymmetric eight with lobes of 337 m and 281 m, the computed capacity was 27 drones. ENHSP solved every count up to 27 and returned no plan for 28, in agreement with the Python check (Table 8.3). With a full 27-drone train, the minimum distance between any two drones at any waypoint step was 9.85 m, including at the crossing. Ten drones with 20 laps solved in 0.8 s, because laps are counted in PDDL and flown on board with a single repeat instruction rather than being unrolled.

Table 8.3: Figure-eight train: ENHSP results on an eight of capacity 27

Drones	2	5	12	25	27	28
Time (s)	0.4	0.4	0.7	2.7	3.3	–
Plan length	26	38	66	118	126	no plan (as intended)

After formation keeping moved on board (Chapter 6), a replay of a logged 16-drone, 2-lap run changed from a median neighbour gap of 23.7 m with a minimum of 0.0 m (collisions) to a median of 10.0 m, with 90 % of samples between 9.2 and 10.8 m, and every drone landed on its own pad.

8.5 Other Plans

Table 8.4 summarises the verification of the remaining plans.

Table 8.4: Verification summary of the remaining plans

Plan	Result
V formation	3-, 5- and 7-drone V's solve through the real pipeline after the neighbour chain was added.
Parabola	27-step plan with a detour, about 0.03–0.1 s. Negative checks (arm at 55 m, flat 50 m line, communication range 15 m) give no plan. In flight, altitudes held at 50/60/60 m, vertex–arm gap 14.1–15.0 m on a straight 1 km route.
Star show	6 drones × 1 lap 0.3 s; 40 drones × 5 laps 0.6 s. In flight (10 drones, 2 laps), worst drone 0.7–3.2 m off its slot while shapes were held, closest pair 9.3 m, every drone landed on its own pad.
Planets	2–8 drones × 3 laps, 0.2–0.3 s. With 8 drones every drone stayed within 0.4 m of its orbit and turned the correct direction and number of turns.
Road inspection	1.4 km and 3.8 km real roads followed with no measurable deviation from the road line; ENHSP 0.3 s; a source 393 m off the road was correctly refused.

8.6 Reading a Parabola Plan

The parabola result follows a clear dependency order. Each drone first passes its checks, takes off and reaches its own cruise height. The planner then establishes the directed links and confirms that every drone can reach its required neighbours. Slot assignment is allowed only after that confirmation. The formation is established, cruises each planned leg, restores its shape at the next location, and finally records mission completion. The selected communication facts in Chapter 5 and the link action in Section 6.2 explain the important parts without repeating the full output.

8.7 Defects Found and Fixed

Verification found several modelling defects. They are recorded here because each one taught a general lesson about writing PDDL for a template-expanding pipeline.

Table 8.5: Modelling defects found during verification

Plan	Defect	Fix and lesson
Grid	`hover-altitude` facts were written over three lines. The line-by-line cloner copied only the middle line, leaving the value undefined for every added drone; AIBR then reported any grid larger than 1×2 as unsolvable.	Rewritten as one line per fact. Templates must keep one fact per line.
Cube	Layer gate written as a comparison of two fluents. A 25-drone, 3-layer cube did not solve in 90 s.	Replaced by propositional layer objects; solved in 1.2 s. Prefer propositional gates for ordering.
Cube	A quantified variable `?l` in `establish-cube-formation` shadowed the action's location parameter, making every problem unsolvable.	Variable renamed. Never reuse a parameter name inside a quantifier.
V formation	Readiness checked only the ground link; there was no notion of which drones sit next to each other.	Added `wing-neighbor` chain and `neighbor-comm-ok`, with `detect-neighbor-link-loss`.
Figure-8	Ground-station correction caused oscillation and collisions in a 16-drone train.	Formation keeping moved on board with a shared pace (Chapter 6).

Chapter 9

Discussion and Limitations

9.1 What the Models Check

An ENHSP plan shows that the written goal can be reached using the allowed actions and the supplied initial facts. The models can check route permission, energy conditions, formation readiness and mission-specific ordering rules. This makes the planning logic visible and reviewable.

The result depends on what is represented. A healthy-link flag is an assumption, not a radio measurement, and a fixed slot is not a physical collision-avoidance controller. The simulator must still execute the route and monitor the drones. A valid symbolic plan therefore does not guarantee that an actual flight will remain safe after unexpected changes.

9.2 Limitations

The current system is useful for studying mission logic, but the following limits must be considered before applying its plans outside the simulator.

- **Fixed cruise height.** A drone generally follows its assigned constant height during cruise, after take-off and climb. Cube layers and parabola arms use different assigned heights. The planner does not choose a changing height profile to follow terrain or avoid new obstacles.
- **No automatic replacement plan.** When a plan fails during execution, the system does not automatically solve a new PDDL problem for the remaining mission. The operator must respond, for example by holding, returning or landing.
- **Mission-specific swarm limits.** Swarm size is configurable for expandable missions, but the current parabola and road-inspection missions remain fixed. The show limits and figure-eight path capacity must also be respected.
- **Symbolic communication.** The PDDL flags describe the assumed state of the links at planning time. They are not updated from live radio measurements during flight; run-time faults are handled by the ground station and the operator, not by replanning.
- **Faults are detected, not planned around.** The detection actions record a failure, but the goals still require healthy drones, so the planner does not produce contingency branches. Online replanning after a fault is left for future work.
- **Battery units.** Battery and energy in PDDL are model units with 100 as full. They are not calibrated against the 5 200 mAh run-time model, so the planning reserve (30 units) and the run-time low threshold (20 %) are separate safeguards.

- **Fault injection coverage.** The GUI offers GPS loss, communications loss and battery failure, but in the thread-based backend only GPS loss is currently delivered to the drones; communication loss is observed from missing messages and low battery arises from consumption.
- **Grounding cost.** Large two-dimensional grids are expensive to ground: a 16-drone (4×4) grid exceeded a 6 GB heap in local testing, even though larger cubes solve, because of the different structure of their quantified conditions.
- **Simulation only.** All results come from the simulator. No physical flight was performed, and the road inspection plan depends on an external routing service.

Chapter 10

Conclusion and Future Work

10.1 Conclusion

This thesis presented the PDDL planning layer of a drone swarm simulator. Ten mission plans were built on one shared vocabulary, covering travel, area search, four rigid formations, a figure-eight train, two shape-changing shows and road inspection. Communication was modelled as neighbour relations with per-drone flags, and, in the parabola plan, as an explicit mesh of range-checked links. Faults were modelled as detection actions with ground-controller responses, and battery as a numeric resource with a return reserve, complemented at run time by thresholds that pause the swarm and ask the operator.

The plans run through the simulator pipeline: they are retargeted to map inputs, solved by ENHSP and flown by the drone threads. Expandable missions use the selected number of drones; the current fixed-size missions retain their stated limits. The results show that careful modelling keeps planning fast for realistic swarm sizes: 64 drones in a cube in 50 s, a 27-drone figure-eight train in 3.3 s, and most missions in well under a second. The defects found during verification led to general modelling rules that will help anyone extending the plans.

10.2 Future Work

1. **Online replanning.** Feed run-time faults back into the PDDL state and replan the remaining mission, for example closing a formation around a lost drone.
2. **Calibrated energy model.** Derive `energy-required` and the reserve from the drone's battery capacity, current draw, speed and wind, so that planning and run-time thresholds use the same units.
3. **A local planner for each drone.** Give every drone its own planner to adjust its remaining route using its local state and nearby drones, while staying consistent with the swarm mission. The present local speed controller follows an existing route; this proposed planner would be able to choose a revised route when conditions change.
4. **Complete fault injection.** Deliver communication loss and battery failure faults to the drone threads, matching the GUI options, and test the operator responses under each.
5. **Larger grids.** Reduce grounding cost for large two-dimensional grids, for example by the same propositional restructuring that made the cube scale.
6. **Physical validation.** Validate the planning assumptions with controlled physical-drone experiments after simulation checks.

References

D. McDermott, M. Ghallab, A. Howe, C. Knoblock, A. Ram, M. Veloso, D. Weld and D. Wilkins, *PDDL – The Planning Domain Definition Language*, Technical Report CVC TR-98-003, Yale Center for Computational Vision and Control, 1998.

M. Fox and D. Long, *PDDL2.1: An Extension to PDDL for Expressing Temporal Planning Domains*, Journal of Artificial Intelligence Research, vol. 20, pp. 61–124, 2003.

M. Ghallab, D. Nau and P. Traverso, *Automated Planning: Theory and Practice*, Morgan Kaufmann, 2004.

E. Scala, P. Haslum, S. Thiébaux and M. Ramirez, *Interval-Based Relaxation for General Numeric Planning*, Proceedings of the 22nd European Conference on Artificial Intelligence (ECAI), pp. 655–663, 2016.

E. Scala, P. Haslum, S. Thiébaux and M. Ramirez, *Subgoaling Techniques for Satisficing and Optimal Numeric Planning*, Journal of Artificial Intelligence Research, vol. 68, pp. 691–752, 2020.

S.-J. Chung, A. A. Paranjape, P. Dames, S. Shen and V. Kumar, *A Survey on Aerial Swarm Robotics*, IEEE Transactions on Robotics, vol. 34, no. 4, pp. 837–855, 2018.

M. Brambilla, E. Ferrante, M. Birattari and M. Dorigo, *Swarm Robotics: A Review from the Swarm Engineering Perspective*, Swarm Intelligence, vol. 7, no. 1, pp. 1–41, 2013.

C. W. Reynolds, *Flocks, Herds and Schools: A Distributed Behavioral Model*, Proceedings of SIGGRAPH '87, Computer Graphics, vol. 21, no. 4, pp. 25–34, 1987.

L. E. Dubins, *On Curves of Minimal Length with a Constraint on Average Curvature, and with Prescribed Initial and Terminal Positions and Tangents*, American Journal of Mathematics, vol. 79, no. 3, pp. 497–516, 1957.

D. Luxen and C. Vetter, *Real-Time Routing with OpenStreetMap Data*, Proceedings of the 19th ACM SIGSPATIAL International Conference on Advances in Geographic Information Systems, pp. 513–516, 2011.

MAVLink Development Team, *MAVLink Micro Air Vehicle Communication Protocol*, `https://mavlink.io/en/`.

E. Scala, *ENHSP – Expressive Numeric Heuristic Search Planner*, `https://sites.google.com/view/enhsp/`.

The Qt Company, *Qt for Python (PySide6)*, `https://doc.qt.io/qtforpython-6/`.